



Machine Learning

CMPE 257

Session 4: Advanced Supervised Learning Techniques

Instructor: Zara Hajihashemi, Ph.D.

Recap

Advanced Supervised Learning

- Decision Trees: Partition the space into regions.
- **Key idea:** Instead of fitting one equation to all the data, we **break the space into pieces** and fit simple predictions inside each piece.
- Decision tree algorithms look for a ***split function*** to define ***regions***.
- Split Function: Misclassification Error, Cross Entropy, Gini Index.
- Regression Tree: Prediction on Continuous values
- CART algorithm

Agenda

- Instance-based Techniques
- Why instance-based?
- K-Nearest Neighbors
- Generative Learning Algorithms
- Gaussian discriminant analysis model
- Naive Bayes
- Unsupervised Learning

Advanced Supervised Learning

Instance-based Techniques

Instance-based Techniques

Motivation

- **Instance-based learning** are the systems that learn the training examples by heart and then generalizes to new instances based on some similarity measure.
- It is called instance-based because it builds the hypotheses from the training instances.
- It is also known as **memory-based learning** or **lazy-learning** (because they delay processing until a new instance must be classified).
- The time complexity of this algorithm depends upon the size of training data.

Instance-based Techniques

Advantages:

1. Instead of estimating for the entire instance set, local approximations can be made to the target function.
2. This algorithm can adapt to new data easily, one which is collected as we go .

Disadvantages:

1. Classification costs are high.
2. Large amount of memory required to store the data, and each query involves starting the identification of a local model from scratch.

Instance-based Techniques

Some of the instance-based learning algorithms are :

1. **K-Nearest Neighbor (KNN)**
2. Self-Organizing Map (SOM)
3. Learning Vector Quantization (LVQ)
4. Locally Weighted Learning (LWL)
5. Case-Based Reasoning

K-Nearest Neighbor (KNN)

Data Representation

- We can represent a single example as a **vector** in a high-dimensional **feature space**.
- Take a data set and map each example to a feature vector through the following mapping:
 1. Real-valued features get copied directly.
 2. Binary features become 0 (for false) or 1 (for true).
 3. Categorical features with V possible values get mapped to V -many binary indicator features.

K-Nearest Neighbor (KNN)

Categorical Data Representation

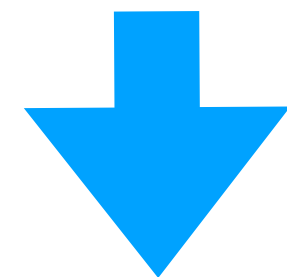
- A categorical feature that can take four different values
 - Red, Blue, Green and Black
- into four binary features
 - IsItRed?, IsItBlue?, IsItGreen? and IsItBlack?
- In general, if we start from a categorical feature that takes V values, we can map it to V -many binary indicator features.

K-Nearest Neighbor (KNN)

Categorical Data Representation

- In general, if we start from a categorical feature that takes V values, we can map it to V -many binary indicator features.
- If you have D -many features (after expanding categorical features), then this **feature vector** will have D -many components.
- This is called “**one-hot encoding**”.
- We will denote feature vectors as $x = \langle x_1, x_2, \dots, x_D \rangle$, so that x_j denotes the value of the j th feature of x .
 - Since these are vectors with real-valued components in D -dimensions, we say that they belong to the space $\mathcal{R}^D$.

Feature: Color
Values: Red, Blue, Green



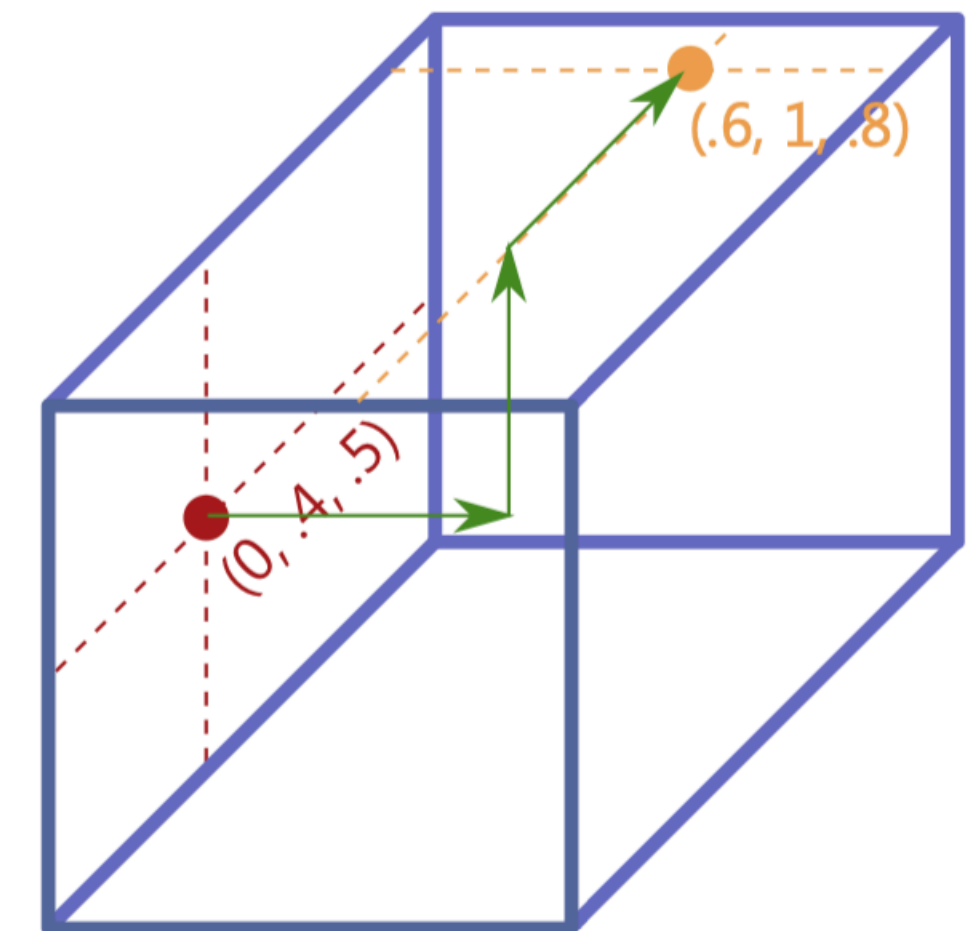
Color_Red	Color_Blue	Color_Green	
1	0	0	← Red
0	1	0	← Blue
0	0	1	← Green

K-Nearest Neighbor (KNN)

Motivation

- The biggest advantage to thinking of examples as vectors in a high dimensional space is that it allows us to apply geometric concepts to machine learning.
- For instance, one of the most basic things that one can do in a vector space is compute **distances**.
- In general, in D -dimensional space, the **Euclidean distance** between vectors a and b is given by:

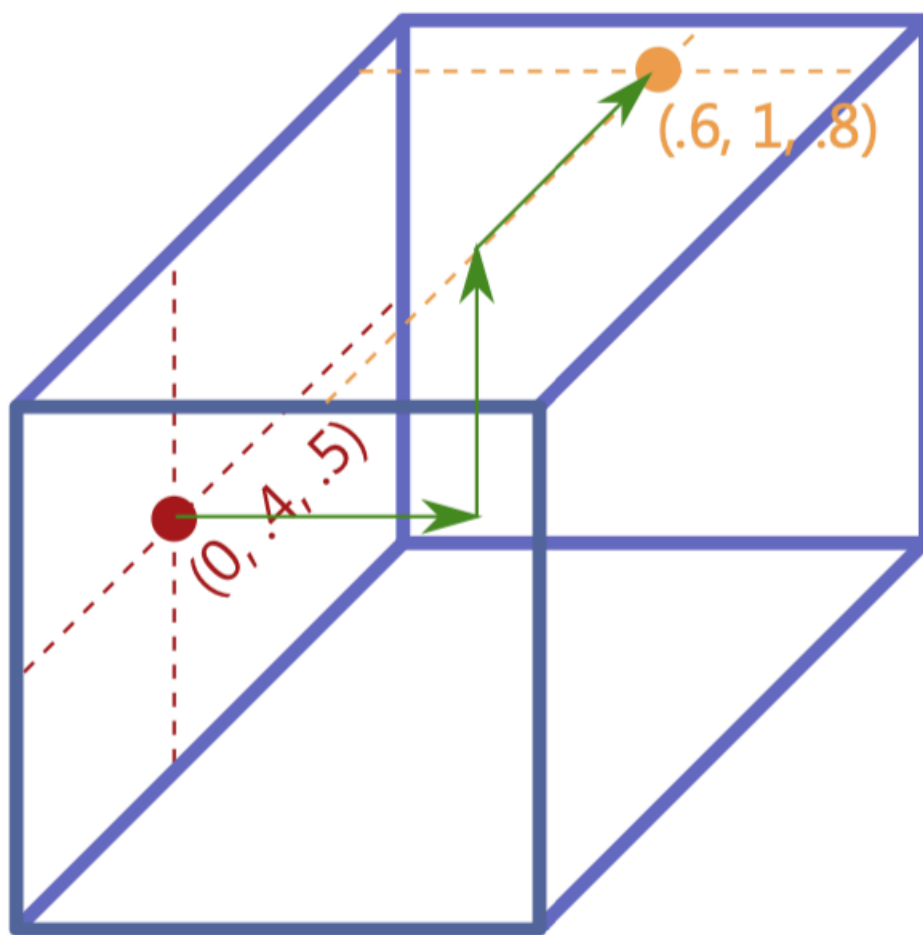
$$d(\mathbf{a}, \mathbf{b}) = \left[\sum_{d=1}^D (a_d - b_d)^2 \right]^{\frac{1}{2}}$$



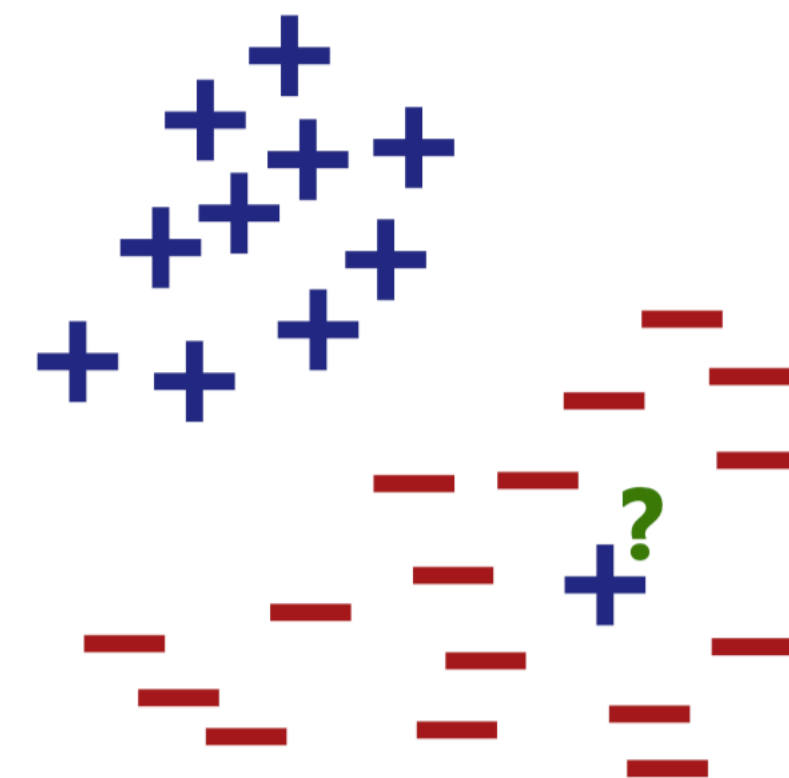
K-Nearest Neighbor (KNN)

Motivation

- Now that we can represent our dataset with vectors and can measure samples with vector distances, we start to think how we can create learning techniques using this new concept?



$$d(\mathbf{a}, \mathbf{b}) = \left[\sum_{d=1}^D (a_d - b_d)^2 \right]^{\frac{1}{2}}$$



K-Nearest Neighbor (KNN)

Algorithm

- The nearest neighbor classifier is build upon **Inductive Bias**.
- **Inductive bias** is the set of assumptions that a machine learning algorithm makes to generalize from observed training data to unseen data.
- The core inductive bias of k-NN is the assumption that similar instances will have similar outputs.
- In other words, data points that are close to each other in the feature space are likely to belong to the same class.

K-Nearest Neighbor (KNN)

Algorithm

- At training time, we simply store the entire training set.
- At test time, we get a test example $\mathbf{x}^{\wedge}$.
- To predict its label, we find the training example $\mathbf{x}$ that is most similar to $\mathbf{x}^{\wedge}$.
- If we consider only one training example, we ran into overfitting problem.
- To solve this, we consider K closet neighbors/samples and let them **vote** on the correct class for this test point

K-Nearest Neighbor (KNN)

Algorithm

Algorithm 3 **KNN-PREDICT**(**D**, **K**, **$\hat{x}$**)

```
1:  $S \leftarrow []$ 
2: for  $n = 1$  to  $N$  do
3:    $S \leftarrow S \oplus \langle d(\mathbf{x}_n, \hat{\mathbf{x}}), n \rangle$            // store distance to training example  $n$ 
4: end for
5:  $S \leftarrow \text{SORT}(S)$                              // put lowest-distance objects first
6:  $\hat{y} \leftarrow 0$ 
7: for  $k = 1$  to  $K$  do
8:    $\langle \text{dist}, n \rangle \leftarrow S_k$                  //  $n$  this is the  $k$ th closest data point
9:    $\hat{y} \leftarrow \hat{y} + y_n$                        // vote according to the label for the  $n$ th training point
10: end for
11: return  $\text{SIGN}(\hat{y})$                              // return  $+1$  if  $\hat{y} > 0$  and  $-1$  if  $\hat{y} < 0$ 
```

D: Training set

N: Number of training examples

K: number of closest samples

$\hat{x}$: Test sample

K-Nearest Neighbor (KNN)

Decision Boundary

- Decision boundaries: KNN leads to flexible, non-linear decision boundaries that are shaped by the local distribution of the training data.
- The resulting boundary is defined by the majority class in every neighborhood, rather than by a predefined mathematical shape like a straight line.

K-Nearest Neighbor (KNN)

K is a hyperparameter

- Small k (e.g., $k=1$):
 - highly sensitive to the local structure of the data and any noise,
 - creating complex decision boundary.
 - Results in a low-bias, high-variance model that can easily overfit the training data.
- Large k:
 - Results in a smoother, more stable decision boundary.
 - This creates a high-bias, low-variance model that leads to underfitting.

k to the square root of the number of samples in the training dataset ($\sqrt{N}$)

K-Nearest Neighbor (KNN)

Example

Training Samples

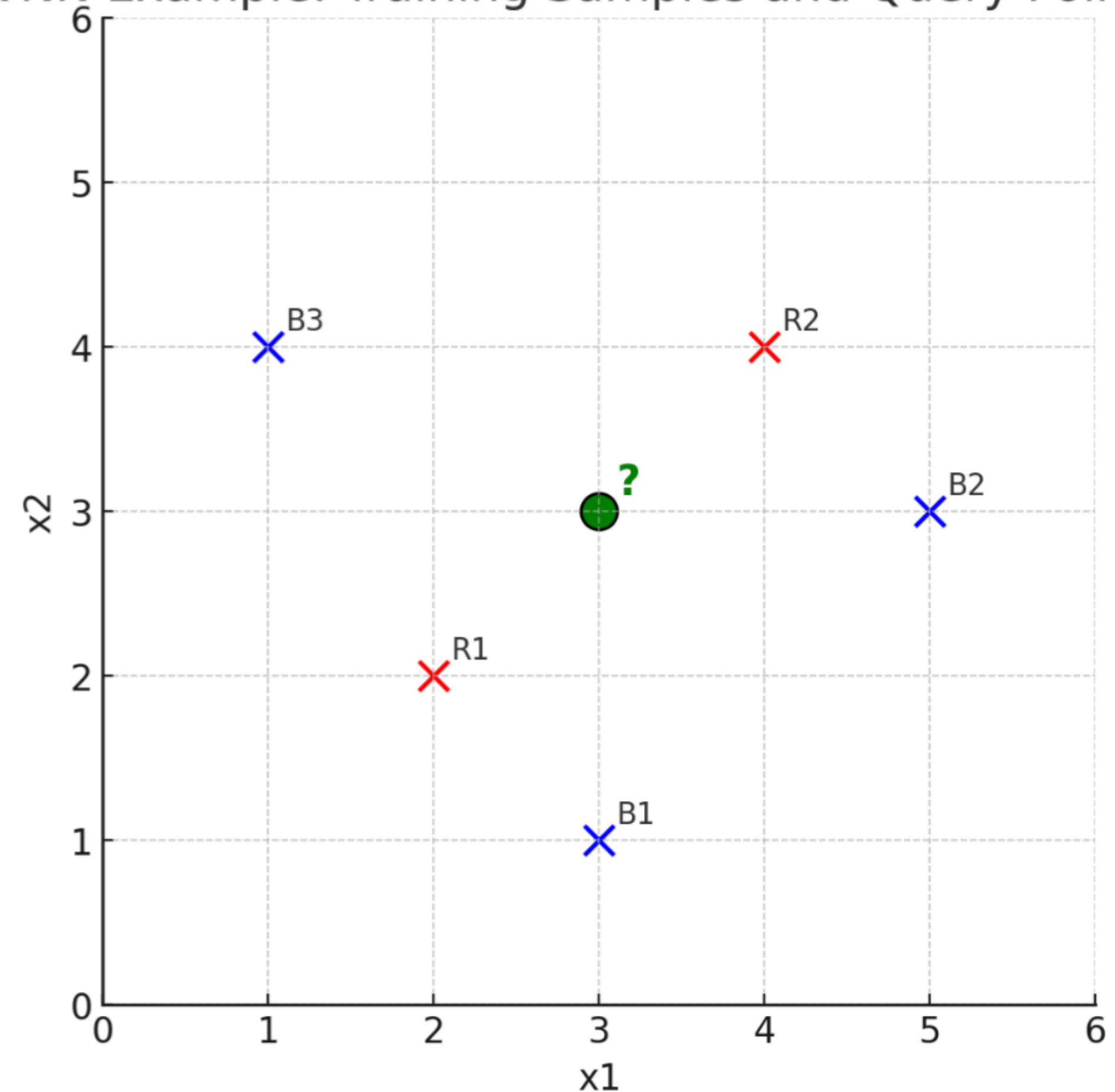
ID	(x1, x2)	Class
R1	(2, 2)	Red
R2	(4, 4)	Red
B1	(3, 1)	Blue
B2	(5, 3)	Blue
B3	(1, 4)	Blue

Query point: $q=(3,3)$

k: 3

Distance: Euclidean

K-NN Example: Training Samples and Query Point (?)



K-Nearest Neighbor (KNN)

Example

Training Samples

ID	(x1, x2)	Class
R1	(2, 2)	Red
R2	(4, 4)	Red
B1	(3, 1)	Blue
B2	(5, 3)	Blue
B3	(1, 4)	Blue

Query point: $q=(3,3)$

k: 3

Distance: Euclidean

Step 1 — Compute distances (exact + numeric)

$$d((x_1, x_2), (u_1, u_2)) = \sqrt{(x_1 - u_1)^2 + (x_2 - u_2)^2}$$

- $d(q, R1) = \sqrt{(3 - 2)^2 + (3 - 2)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, R2) = \sqrt{(3 - 4)^2 + (3 - 4)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, B1) = \sqrt{(3 - 3)^2 + (3 - 1)^2} = \sqrt{0 + 4} = 2.000$
- $d(q, B2) = \sqrt{(3 - 5)^2 + (3 - 3)^2} = \sqrt{4 + 0} = 2.000$
- $d(q, B3) = \sqrt{(3 - 1)^2 + (3 - 4)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.236$

K-Nearest Neighbor (KNN)

Example

Training Samples

ID	(x1, x2)	Class
R1	(2, 2)	Red
R2	(4, 4)	Red
B1	(3, 1)	Blue
B2	(5, 3)	Blue
B3	(1, 4)	Blue

Query point: $q=(3,3)$

k: 3

Distance: Euclidean

Step 1 — Compute distances (exact + numeric)

$$d((x_1, x_2), (u_1, u_2)) = \sqrt{(x_1 - u_1)^2 + (x_2 - u_2)^2}$$

- $d(q, R1) = \sqrt{(3 - 2)^2 + (3 - 2)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, R2) = \sqrt{(3 - 4)^2 + (3 - 4)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, B1) = \sqrt{(3 - 3)^2 + (3 - 1)^2} = \sqrt{0 + 4} = 2.000$
- $d(q, B2) = \sqrt{(3 - 5)^2 + (3 - 3)^2} = \sqrt{4 + 0} = 2.000$
- $d(q, B3) = \sqrt{(3 - 1)^2 + (3 - 4)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.236$

Step 2 — Sort neighbors by distance

1. R1: 1.414 (Red)
2. R2: 1.414 (Red)
3. B1: 2.000 (Blue)
4. B2: 2.000 (Blue)
5. B3: 2.236 (Blue)

K-Nearest Neighbor (KNN)

Example

Training Samples

ID	(x1, x2)	Class
R1	(2, 2)	Red
R2	(4, 4)	Red
B1	(3, 1)	Blue
B2	(5, 3)	Blue
B3	(1, 4)	Blue

Query point: $q=(3,3)$

k : 3

Distance: Euclidean

Step 1 — Compute distances (exact + numeric)

$$d((x_1, x_2), (u_1, u_2)) = \sqrt{(x_1 - u_1)^2 + (x_2 - u_2)^2}$$

- $d(q, R1) = \sqrt{(3 - 2)^2 + (3 - 2)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, R2) = \sqrt{(3 - 4)^2 + (3 - 4)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, B1) = \sqrt{(3 - 3)^2 + (3 - 1)^2} = \sqrt{0 + 4} = 2.000$
- $d(q, B2) = \sqrt{(3 - 5)^2 + (3 - 3)^2} = \sqrt{4 + 0} = 2.000$
- $d(q, B3) = \sqrt{(3 - 1)^2 + (3 - 4)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.236$

Step 2 — Sort neighbors by distance

1. R1: 1.414 (Red)
2. R2: 1.414 (Red)
3. B1: 2.000 (Blue)
4. B2: 2.000 (Blue)
5. B3: 2.236 (Blue)

Step 3 — Take the nearest $k = 3$

- Neighbors: R1 (Red), R2 (Red), B1 (Blue)
- Vote: Red = 2, Blue = 1 → Predict: Red

Weighted K-Nearest Neighbor (KNN)

Intuition

Give closer neighbors more influence with weight $w_i = \frac{1}{d_i + \epsilon}$

Using $\epsilon = 10^{-6}$ for stability

K-Nearest Neighbor (KNN)

Example

Training Samples

ID	(x1, x2)	Class
R1	(2, 2)	Red
R2	(4, 4)	Red
B1	(3, 1)	Blue
B2	(5, 3)	Blue
B3	(1, 4)	Blue

Query point: $q=(3,3)$

k: 3

Distance: Euclidean

Step 1 — Compute distances (exact + numeric)

$$d((x_1, x_2), (u_1, u_2)) = \sqrt{(x_1 - u_1)^2 + (x_2 - u_2)^2}$$

- $d(q, R1) = \sqrt{(3 - 2)^2 + (3 - 2)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, R2) = \sqrt{(3 - 4)^2 + (3 - 4)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.414$
- $d(q, B1) = \sqrt{(3 - 3)^2 + (3 - 1)^2} = \sqrt{0 + 4} = 2.000$
- $d(q, B2) = \sqrt{(3 - 5)^2 + (3 - 3)^2} = \sqrt{4 + 0} = 2.000$
- $d(q, B3) = \sqrt{(3 - 1)^2 + (3 - 4)^2} = \sqrt{4 + 1} = \sqrt{5} \approx 2.236$

Step 2 — Sort neighbors by distance

1. R1: 1.414 (Red)
2. R2: 1.414 (Red)
3. B1: 2.000 (Blue)
4. B2: 2.000 (Blue)
5. B3: 2.236 (Blue)

- Weights for the top 3:

$$w_{R1} \approx 0.707, w_{R2} \approx 0.707, w_{B1} = 0.5$$

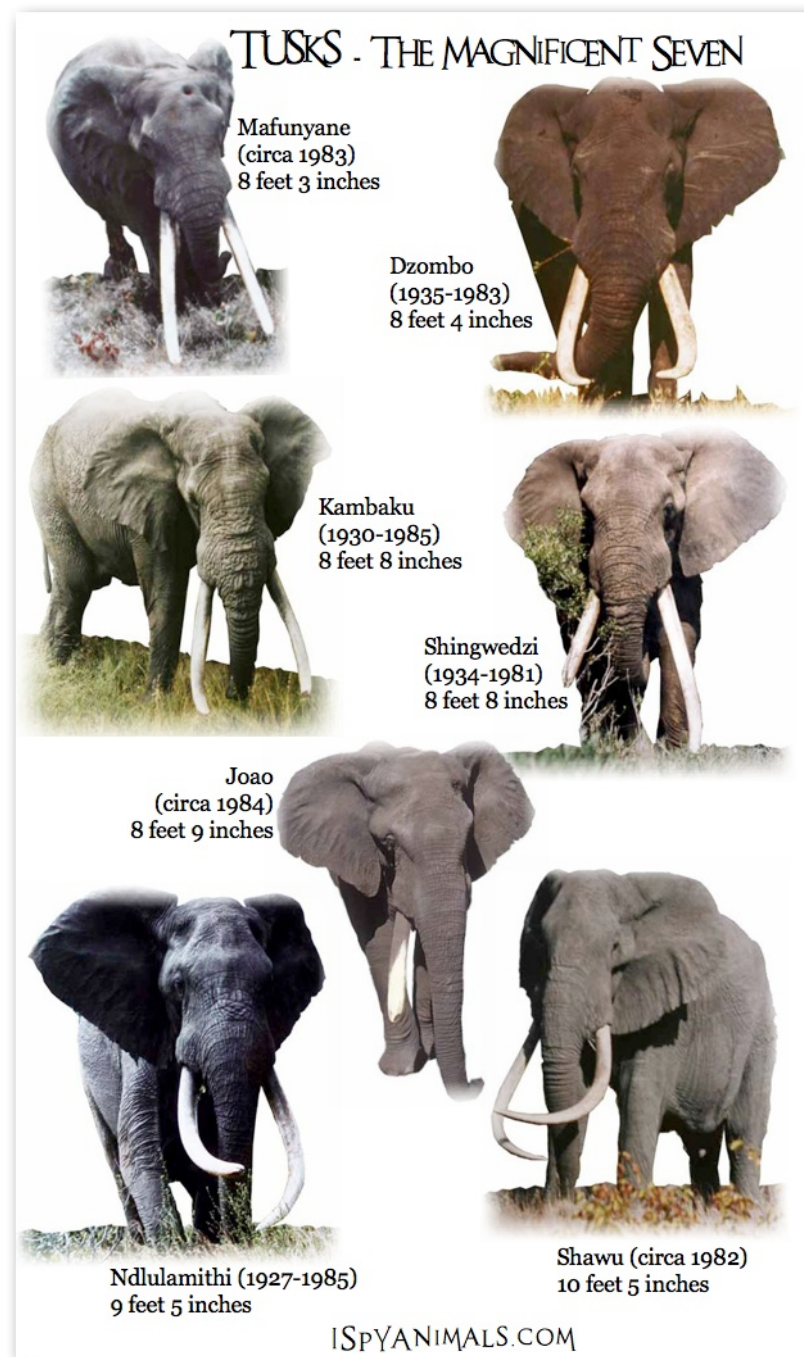
Weighted vote still favors **Red** ($0.707 + 0.707 > 0.5$).

Generative Learning Algorithms

Generative vs. Discriminative Models

Motivation

- **Discriminative:** model $p(y|x)$ directly



$(y = 1)$

VS.



$(y = 0)$

1. Logistic regression or the perceptron algorithm: tries to find a straight line—a decision boundary.

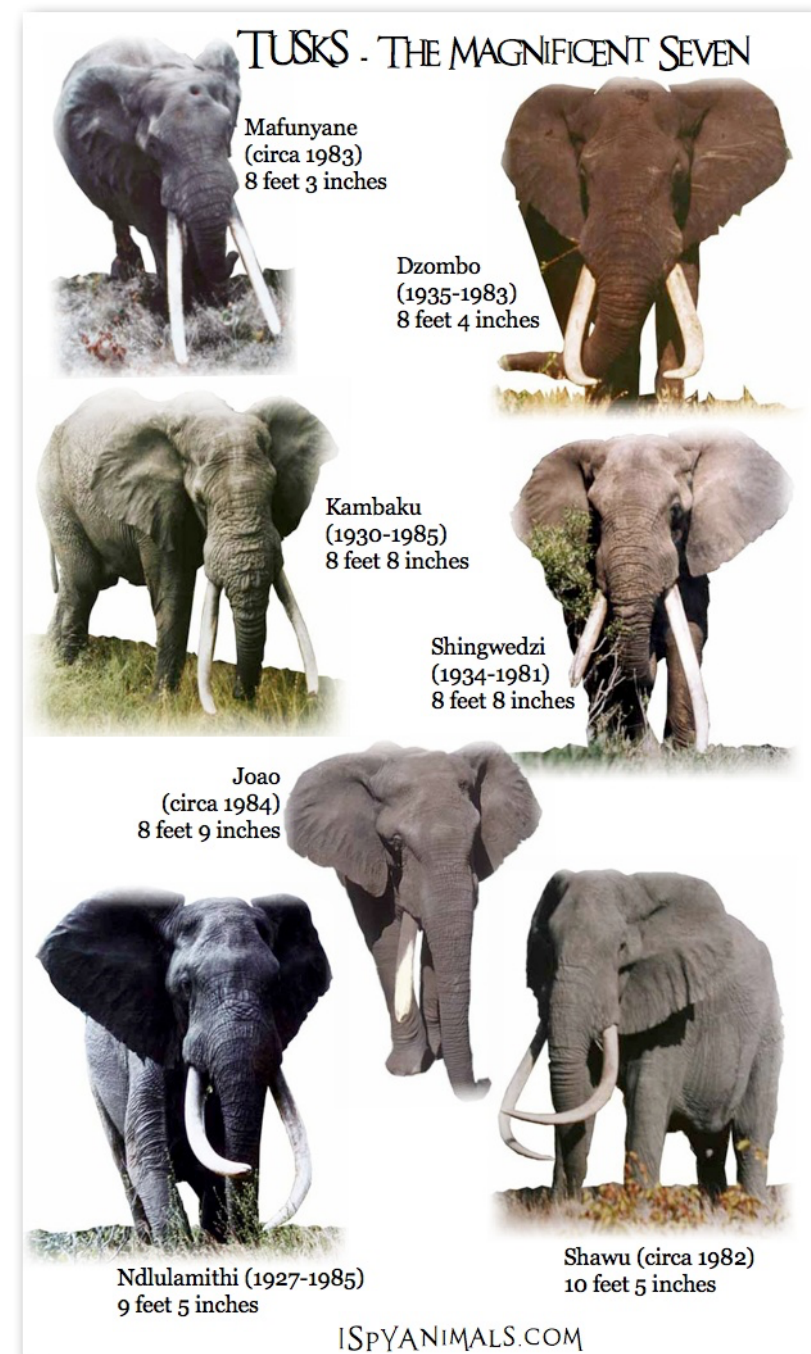
2. learn $p(y|x)$

First, looking at elephants, we can build a model of what elephants look like. Then, looking at dogs, we can build a separate model of what dogs look like. Finally, to classify a new animal, we can match the new animal against the elephant model, and match it against the dog model, to see whether the new animal looks more like the elephants or more like the dogs we had seen in the training set.

Generative vs. Discriminative Models

Motivation

- **Generative:** learn joint distribution $p(x, y)$; derive $p(y|x)$ via Bayes



$(y = 1)$

VS.



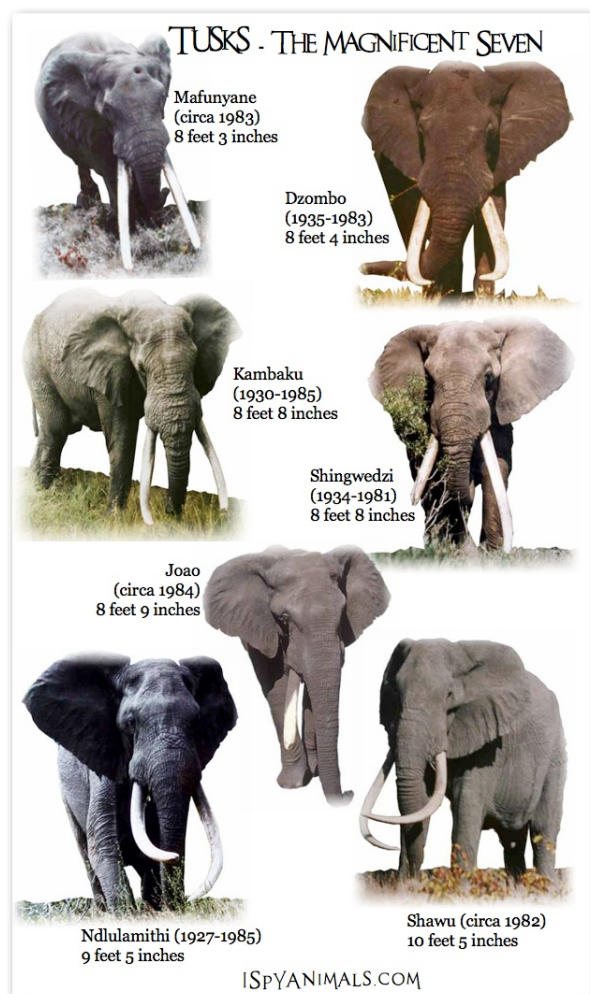
$(y = 0)$

if y indicates whether an example is a dog (0) or an elephant (1),
then $p(x|y = 0)$ models the distribution of dogs' features, and
 $p(x|y = 1)$ models the distribution of elephants' features.

Generative vs. Discriminative Models

Motivation

- **Generative vs. Discriminative**


$$(y = 1)$$

VS.


$$(y = 0)$$

Generative: learn/model the joint distribution $p(x, y)$, typically by modeling $p(x \mid y)$ and $p(y)$; then derive $p(y \mid x)$ using Bayes' rule.

Because:

and then:

$$p(x, y) = p(x \mid y)p(y)$$

$$p(y \mid x) = \frac{p(x \mid y)p(y)}{p(x)}$$

Generative vs. Discriminative Models

Motivation

- **Generative vs. Discriminative**

- **Generative:** model $p(x, y) = p(x | y)p(y) \rightarrow$ derive $p(y | x)$ using Bayes.
- **Discriminative:** directly model $p(y | x)$.

Generative vs. Discriminative Models

Definition

- **Generative:** learn joint distribution $p(x, y)$; derive $p(y|x)$ via Bayes

After modeling

Class priors $p(y = 0)$, and

$p(x|y = 1)$;

use Bayes rule to derive the posterior distribution on y given x :

$$p(y|x) = \frac{p(x|y)p(y)}{p(x)}$$

$$p(x) = p(x|y = 1)p(y = 1) + p(x|y = 0)p(y = 0)$$

$$\begin{aligned}\arg \max_y p(y|x) &= \arg \max_y \frac{p(x|y)p(y)}{p(x)} \\ &= \arg \max_y p(x|y)p(y).\end{aligned}$$

Generative vs. Discriminative Models

1. Models based on $P(y \mid x)$: Discriminative models

These models directly answer:

Given the features x , what is the probability of class y ?

$$P(y \mid x)$$

For example, suppose you're predicting whether a patient has cancer based on test results:

$$x = \{\text{age, biomarker level, imaging features}\}$$

$$y \in \{\text{Cancer, No Cancer}\}$$

A discriminative model directly estimates:

$$P(\text{Cancer} \mid x)$$

For example:

$$P(\text{Cancer} \mid x) = 0.85$$

and then predicts Cancer.

Examples: Logistic Regression, neural-network classifiers, and many other classifiers that directly learn a decision boundary or conditional probability.

Generative vs. Discriminative Models

2. Models based on $P(x | y)$: Generative models

These approach the problem from the opposite direction:

If I know the class y , how likely am I to observe these features x ?

$$P(x | y)$$

For example:

$$P(x | \text{Cancer})$$

asks:

How likely would these particular test results be if this patient had cancer?

The model separately learns something like:

$$P(x | \text{Cancer})$$

and

$$P(x | \text{No Cancer})$$

Examples:

Naive Bayes,
Gaussian Discriminant Analysis (GDA),
LDA/QDA when viewed probabilistically

Generative vs. Discriminative Models

	$P(y x)$	$P(x y)$
Question	"Given x, what is y?"	"Given y, what would x look like?"
Type	Discriminative	Generative
Learns	Class probability directly	Distribution of features within each class
Uses Bayes rule?	No	Yes
Example	Logistic Regression	Naive Bayes
Models distribution of x?	No	Yes

Gaussian discriminant analysis

Definition

The first generative learning algorithm that we'll look at is **Gaussian discriminant analysis (GDA)**.

In this model, we'll assume that $p(y|x)$ is distributed according to a multivariate normal distribution.

Multivariate normal distribution in d -dimensions
(multivariate Gaussian distribution) is parameterized by

mean vector $\mu \in \mathbb{R}^d$
a covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$, where $\Sigma \geq 0$

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

Gaussian discriminant analysis

Definition

Multivariate normal distribution in d -dimensions
(multivariate Gaussian distribution) is parameterized by

mean vector $\mu \in \mathbb{R}^d$
a covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$, where $\Sigma \geq 0$

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

For a random variable X distributed $\mathcal{N}(\mu, \Sigma)$, the mean is: $\mathbb{E}[X] = \int_x x p(x; \mu, \Sigma) dx = \mu$

The *covariance* of a vector-valued random variable Z : $\text{Cov}(Z) = \mathbb{E}[(Z - \mathbb{E}[Z])(Z - \mathbb{E}[Z])^T]$

$$\text{Cov}(Z) = \mathbb{E}[ZZ^T] - (\mathbb{E}[Z])(\mathbb{E}[Z])^T$$

Gaussian discriminant analysis

Intuition

For a random variable $\mathcal{X}$ distributed $\mathcal{N}(\mu, \Sigma)$:

- **μ (the mean)** tells you the *center* of the cloud of points (the average location).
- **Σ (the covariance matrix)** tells you the *shape and spread* of that cloud.
- **The diagonal values of Σ** show how much each feature (dimension of $\mathcal{X}$) varies on its own.
(Big numbers = more spread out).
- **The off-diagonal values show how two features move together:**
 - Positive covariance $\rightarrow$ as one goes up, the other tends to go up.
 - Negative covariance $\rightarrow$ as one goes up, the other tends to go down.
 - Zero covariance $\rightarrow$ no particular relationship.

Gaussian discriminant analysis

Intuition

The key is that **Gaussian Discriminant Analysis (GDA)**, despite having “**Discriminant**” in its name, is a **generative classifier**.

Why is it called “Discriminant Analysis” then?

The word **discriminant** in *Gaussian Discriminant Analysis* refers to the fact that we ultimately construct a **discriminant function / decision boundary** to discriminate between classes. It does **not** mean that GDA belongs to the category called *discriminative models*.

Gaussian discriminant analysis

Why is it called “Discriminant Analysis” then?

GDA assumes:

$$y \sim \text{Bernoulli}(\phi)$$

and then models how the data are generated within each class:

$$x \mid y = 0 \sim \mathcal{N}(\mu_0, \Sigma)$$

$$x \mid y = 1 \sim \mathcal{N}(\mu_1, \Sigma)$$

So it learns:

$$P(y) \quad \text{and} \quad P(x \mid y)$$

That makes it a **generative model**.

Then, when you give GDA a new x , it uses Bayes' rule:

$$P(y \mid x) = \frac{P(x \mid y)P(y)}{P(x)}$$

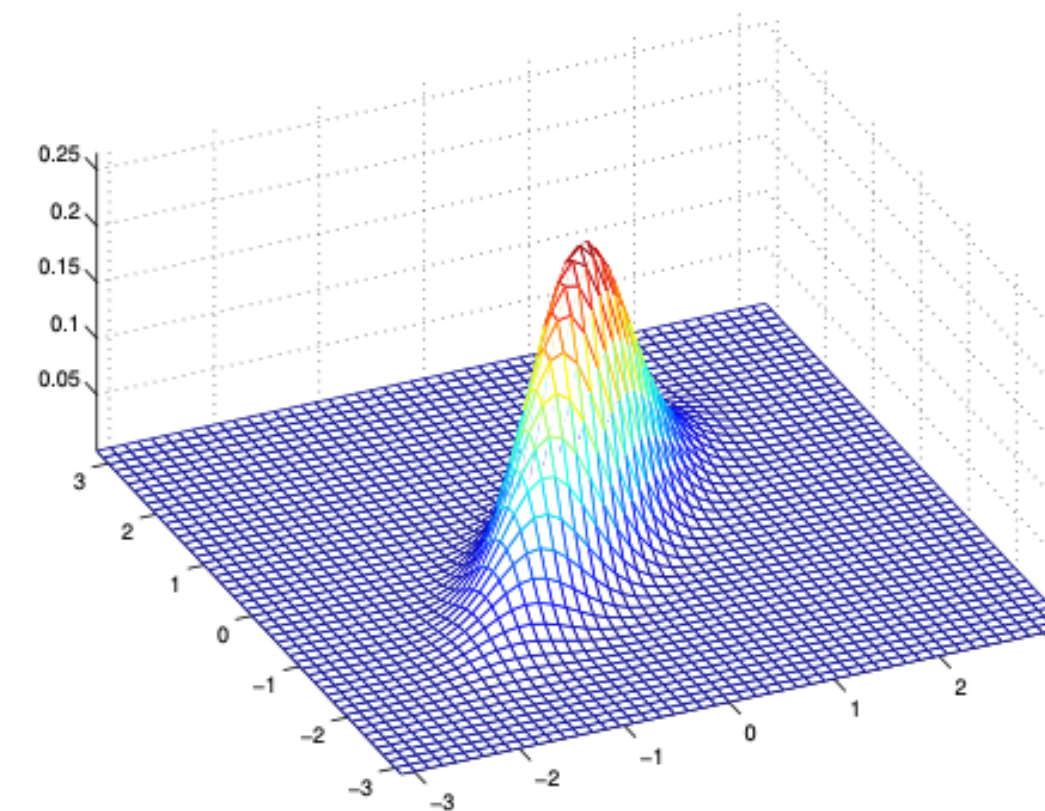
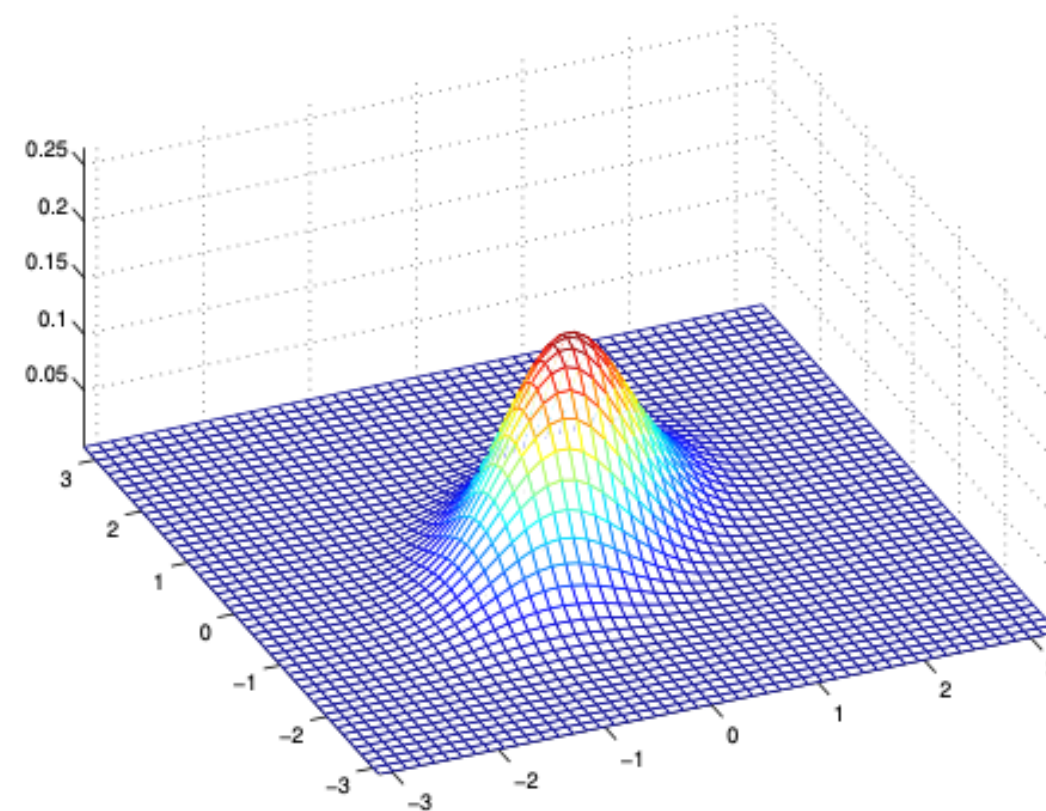
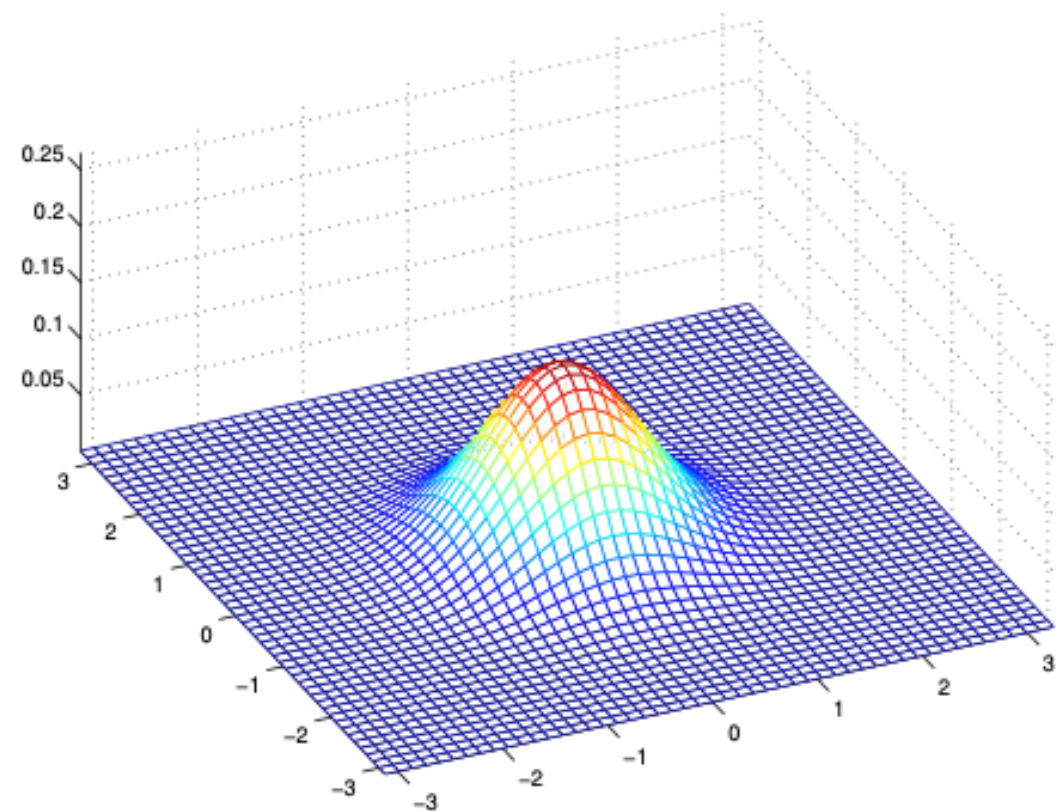
and classifies based on which class has the larger posterior probability.

Gaussian discriminant analysis

Intuition

For a random variable $\mathcal{X}$ distributed $\mathcal{N}(\mu, \Sigma)$:

- μ (the mean) tells you the *center* of the cloud of points (the average location).
- Σ (the covariance matrix) tells you the *shape and spread* of that cloud.

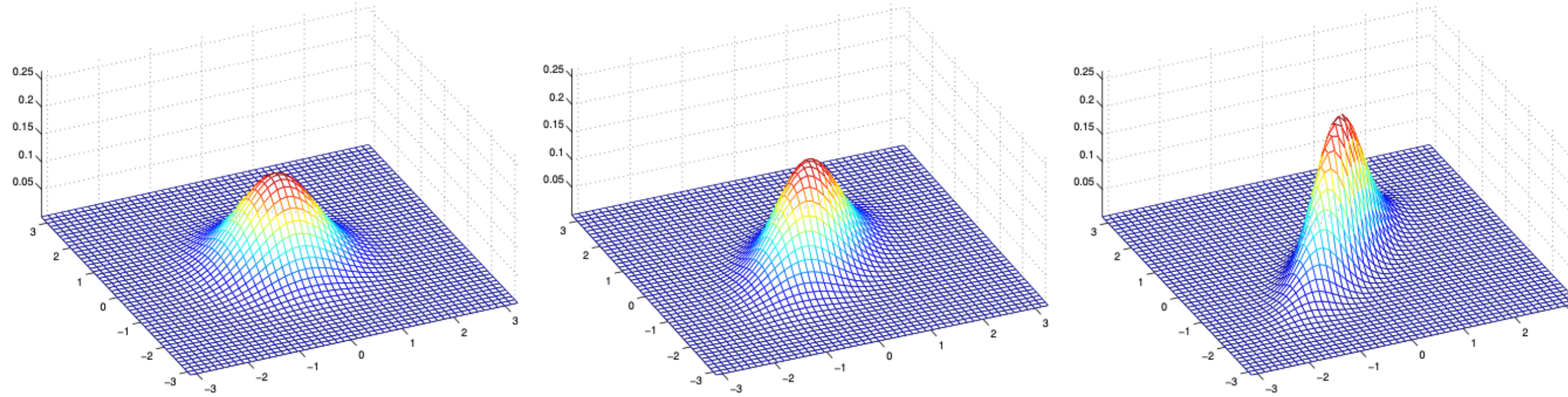


$\mu = 0$

$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$

Gaussian discriminant analysis

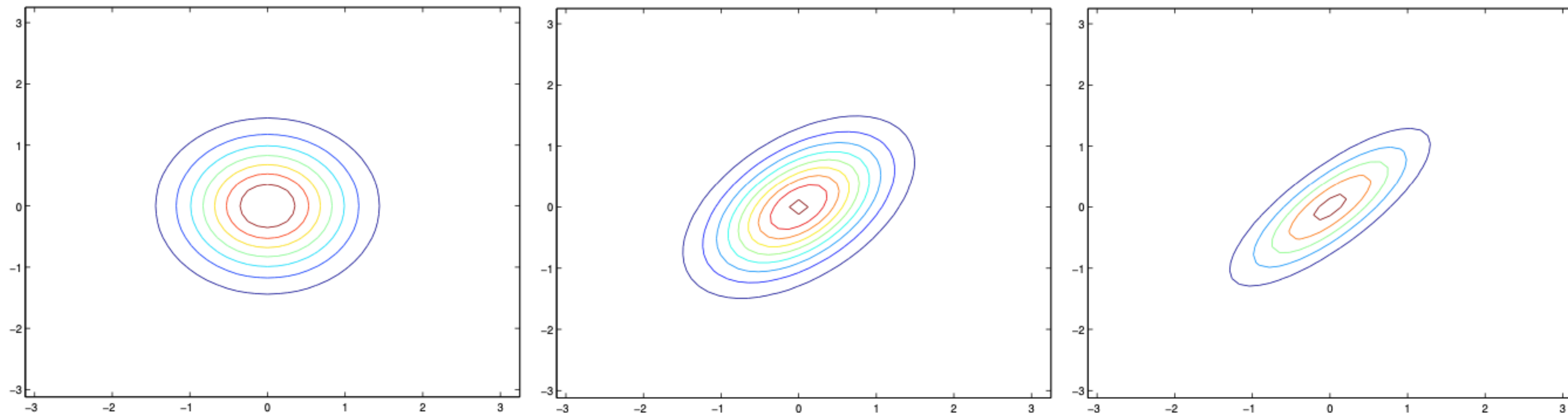
Intuition



Looking at the contours of the same three densities

$$\mu = 0$$

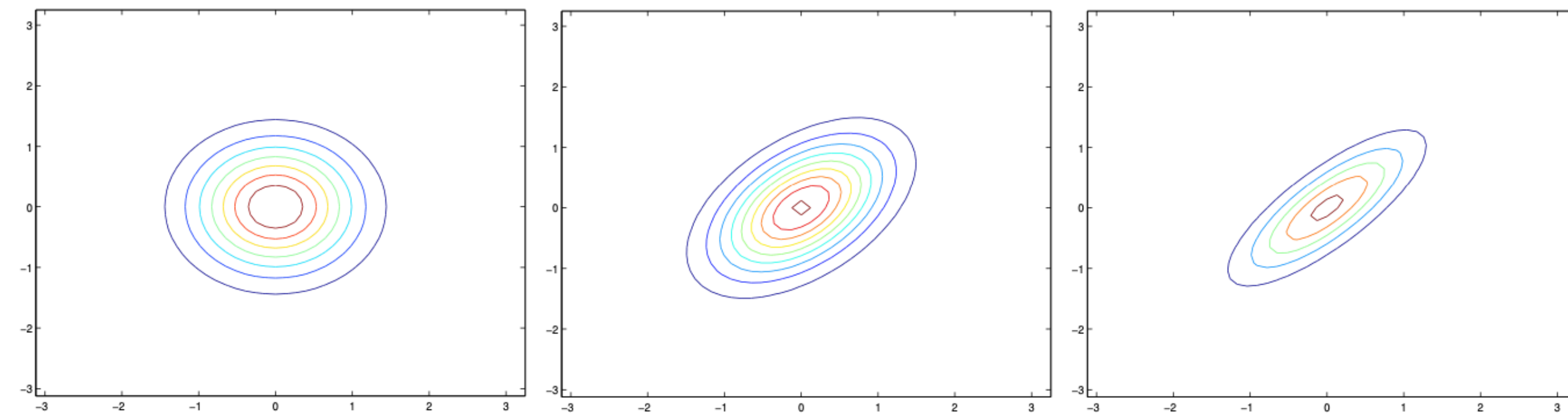
$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$



Gaussian discriminant analysis

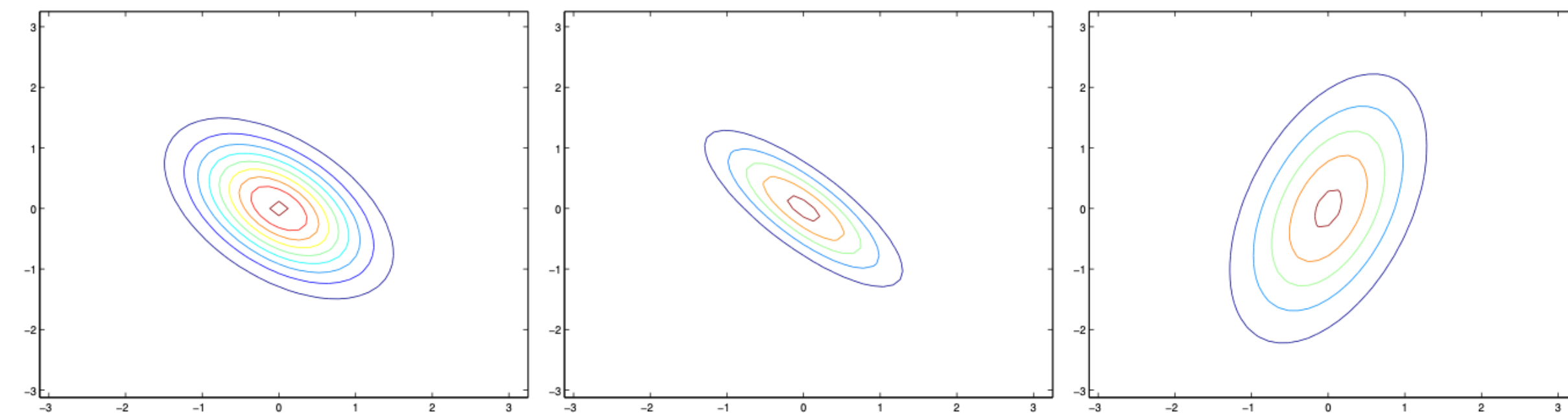
Intuition

$$\Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$



$\mu = 0$

$$\Sigma = \begin{bmatrix} 1 & -0.5 \\ -0.5 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 1 & -0.8 \\ -0.8 & 1 \end{bmatrix}; \quad \Sigma = \begin{bmatrix} 3 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$



$\mu = 0$

Gaussian Discriminant Analysis

Formal Definition

When we have a classification problem in which the input features x are continuous-valued random variables, we can then use the **Gaussian Discriminant Analysis (GDA)** model, which models $p(x|y)$ using a multivariate normal distribution. The model is:

$$\begin{aligned} y &\sim \text{Bernoulli}(\phi) \\ x|y = 0 &\sim \mathcal{N}(\mu_0, \Sigma) \\ x|y = 1 &\sim \mathcal{N}(\mu_1, \Sigma) \end{aligned}$$

Expanding distributions we have:

$$\begin{aligned} p(y) &= \phi^y (1 - \phi)^{1-y} \\ p(x|y = 0) &= \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_0)^T \Sigma^{-1} (x - \mu_0) \right) \\ p(x|y = 1) &= \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_1)^T \Sigma^{-1} (x - \mu_1) \right) \end{aligned}$$

Where the parameters of our model are ϕ , Σ , mean vectors $[\mu_0 \text{ and } \mu_1]$

Gaussian Discriminant Analysis

Formal Definition

When we have a classification problem in which the input features x are continuous-valued random variables,

we can then use the **Gaussian Discriminant Analysis (GDA)** model, which models $p(x|y)$ using a multivariate normal distribution.

The model is:

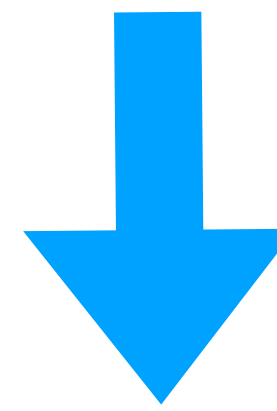
$$\begin{aligned} y &\sim \text{Bernoulli}(\phi) \\ x|y = 0 &\sim \mathcal{N}(\mu_0, \Sigma) \\ x|y = 1 &\sim \mathcal{N}(\mu_1, \Sigma) \end{aligned}$$

Gaussian Discriminant Analysis

Formal Definition

$$\begin{aligned}y &\sim \text{Bernoulli}(\phi) \\x|y=0 &\sim \mathcal{N}(\mu_0, \Sigma) \\x|y=1 &\sim \mathcal{N}(\mu_1, \Sigma)\end{aligned}$$

ϕ : the parameter that represents the probability of success in a single trial



$$\begin{aligned}p(y) &= \phi^y (1 - \phi)^{1-y} \\p(x|y=0) &= \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_0)^T \Sigma^{-1} (x - \mu_0) \right) \\p(x|y=1) &= \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (x - \mu_1)^T \Sigma^{-1} (x - \mu_1) \right)\end{aligned}$$

Gaussian Discriminant Analysis

Formal Definition

$$\begin{aligned}p(y) &= \phi^y(1 - \phi)^{1-y} \\p(x|y = 0) &= \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0)\right) \\p(x|y = 1) &= \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)\right)\end{aligned}$$

The log-likelihood of the data is given by:

$$\begin{aligned}\ell(\phi, \mu_0, \mu_1, \Sigma) &= \log \prod_{i=1}^n p(x^{(i)}, y^{(i)}; \phi, \mu_0, \mu_1, \Sigma) \\&= \log \prod_{i=1}^n p(x^{(i)}|y^{(i)}; \mu_0, \mu_1, \Sigma)p(y^{(i)}; \phi).\end{aligned}$$

Gaussian Discriminant Analysis

Formal Definition

By maximizing ℓ with respect to the parameters, we find the maximum likelihood estimate of the parameters to be:

class prior: the probability that a random example belongs to class 1

$$\phi = \frac{1}{n} \sum_{i=1}^n 1\{y^{(i)} = 1\}$$

mean vector (average values of features) for all examples where $\mathbf{y} = \mathbf{0}$ (class 0).

$$\mu_0 = \frac{\sum_{i=1}^n 1\{y^{(i)} = 0\} x^{(i)}}{\sum_{i=1}^n 1\{y^{(i)} = 0\}}$$

mean vector (average values of features) for all examples where $\mathbf{y} = \mathbf{1}$ (class 1).

$$\mu_1 = \frac{\sum_{i=1}^n 1\{y^{(i)} = 1\} x^{(i)}}{\sum_{i=1}^n 1\{y^{(i)} = 1\}}$$

covariance matrix, shared across both classes;

It captures the spread and relationships between features.

Σ tells us the **shape of the data cloud** (round, stretched, tilted) that applies to both classes.

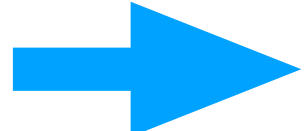
$$\Sigma = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

Naive bayes

Naive Bayes

Intuition

In GDA, the feature vectors x were continuous, real-valued vectors. Let's see what happens when the x_j 's are discrete valued.

Example: email spam filter  ***Text Classification***

Data Representation:

- We will represent an email via a feature vector whose length is equal to *the number of words in the dictionary*.
- If an email contains the j^{th} word of the dictionary, then we will set $x_j = 1$; otherwise, we let $x_j = 0$.

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{array}{l} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{array}$$

Naive Bayes

Intuition

Modeling:

- But if we have, say, a vocabulary of 50000 words,
- then $x \in \{0, 1\}^{50000}$ (x is a 50000-dimensional vector of 0's and 1's), and
- if we were to model x explicitly with a multinomial distribution over the 2^{50000} possible outcomes, then
- we'd end up with a $(2^{50000} - 1)$ dimensional parameter vector.
- This is clearly too many parameters.

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \begin{matrix} \text{a} \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

Naive Bayes

Assumption

Modeling assumption:

- To model $p(y|x)$, we will therefore make a very strong assumption.
We will assume that the x 's are conditionally independent given y .
This assumption is called the **Naive Bayes (NB) assumption**, and
- This means the presence or absence of one feature doesn't influence the presence or absence of another feature, considering the class is already known.
- The resulting algorithm is called the **Naive Bayes classifier**.
- In reality, this assumption isn't always true.

Naive Bayes

Assumption

Modeling assumption:

- **Bayes' theorem**, a theorem in probability theory that helps to calculate the **conditional probability** of an event (the class label) happening, given another event (the features) has already occurred.
- The resulting algorithm is called the **Naive Bayes classifier**.

BAYES' THEOREM

THE PROBABILITY OF A HYPOTHESIS, **H**
CONDITIONAL ON A NEW PIECE OF EVIDENCE, **E**

PROBABILITY OF THE EVIDENCE GIVEN THE HYPOTHESIS

THE PRIOR PROBABILITY OF THE HYPOTHESIS

$$P(H | E) = \frac{P(E | H) P(H)}{P(E)}$$

THE PRIOR PROBABILITY OF THE EVIDENCE

Naive Bayes

Assumption

**Given a features vector $X=(x_1, x_2, \dots, x_n)$ and a class variable y ,
Bayes Theorem states that:**

$$x = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad \begin{matrix} a \\ \text{aardvark} \\ \text{aardwolf} \\ \vdots \\ \text{buy} \\ \vdots \\ \text{zygmurgy} \end{matrix}$$

$$P(y|X) = \frac{P(X|y) * P(y)}{P(X)}$$

the posterior probability

the likelihood

prior probabilities

Naive Bayes

Assumption

Using the Chain rule the likelihood $p(x|y)$ can be written as:

$$\begin{aligned} P(X|y) &= P(x_1, x_2, \dots, x_n | y) \\ &= P(x_1 | x_2, \dots, x_n, y) * P(x_2 | x_3, \dots, x_n, y) \dots P(x_n | y) \end{aligned}$$

Because of Naive assumption on conditional independence we can have:

$$P(X|y) = P(x_1|y) * P(x_2|y) \dots P(x_n|y)$$

Naive Bayes

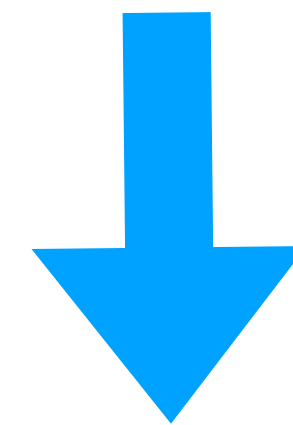
Assumption

$$P(X|y) = P(x_1, x_2, \dots, x_n|y)$$

$$= P(x_1|x_2, \dots, x_n, y) * P(x_2|x_3, \dots, x_n, y) \dots P(x_n|y)$$

$$(1) \quad P(X|y) = P(x_1|y) * P(x_2|y) \dots P(x_n|y)$$

$$(2) \quad P(y|X) = \frac{P(X|y) * P(y)}{P(X)}$$



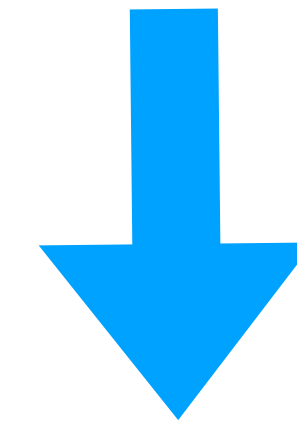
$$P(y|X) = \frac{P(x_1|y) * P(x_2|y) \dots P(x_n|y) * P(y)}{P(x_1) * P(x_2) \dots P(x_n)}$$

Naive Bayes

Assumption

$$P(y|X) = \frac{P(x_1|y) * P(x_2|y) \dots\dots P(x_n|y) * P(y)}{P(x_1) * P(x_2) \dots\dots P(x_n)}$$

And as denominator remains constant for all values, the posterior probability can then be:



$$P(y|x_1, x_2, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

Naive Bayes

$$P(y|x_1, x_2, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y)$$

The Naive Bayes classifier combines this model with a decision rule.

One common rule is to pick the hypothesis that's most probable.

this is known as the **maximum a posteriori or MAP decision rule**.

Naive Bayes

Formal Definition

Model parameter:

$$\left\{ \begin{array}{l} \phi_{j|y=1} = p(x_j = 1 | y = 1) \\ \phi_{j|y=0} = p(x_j = 1 | y = 0) \\ \phi_y = p(y = 1) \end{array} \right.$$

Given a training set $\Rightarrow \{(x^{(i)}, y^{(i)}); i = 1, \dots, n\}$

class prior: the probability that a random example belongs to a class

The joint likelihood of the data: $\mathcal{L}(\phi_y, \phi_{j|y=0}, \phi_{j|y=1}) = \prod_{i=1}^n p(x^{(i)}, y^{(i)})$

Naive Bayes

Assumptions

The fundamental Naive Bayes assumption is that each feature makes an:

- **Feature independence:** The features of the data are conditionally independent of each other, given the class label.
- **Continuous features are normally distributed:** If a feature is continuous, then it is assumed to be normally distributed within each class.
- **Discrete features have multinomial distributions:** If a feature is discrete, then it is assumed to have a multinomial distribution within each class.
- **Features are equally important:** All features are assumed to contribute equally to the prediction of the class label.
- **No missing data:** The data should not contain any missing values.

Naive Bayes

Prediction with NB

To make a prediction on a new example with features x , we then simply calculate:

$$\begin{aligned} p(y = 1|x) &= \frac{p(x|y = 1)p(y = 1)}{p(x)} \\ &= \frac{\left(\prod_{j=1}^d p(x_j|y = 1)\right) p(y = 1)}{\left(\prod_{j=1}^d p(x_j|y = 1)\right) p(y = 1) + \left(\prod_{j=1}^d p(x_j|y = 0)\right) p(y = 0)} \end{aligned}$$

And pick whichever class has the higher posterior probability.

Naive Bayes

Prediction with NB

```
NaiveBayesTrain(Dataset):  
    For each class y:  
        Compute prior[y] = count(y) / total_examples  
        For each feature j:  
            Estimate conditional[j][y] = probability(x_j | y)  
    Return model(prior, conditional)  
  
NaiveBayesPredict(model, x):  
    For each class y:  
        score[y] = prior[y]  
        For each feature j:  
            score[y] *= conditional[j][y]  
    Return class with max(score[y])
```

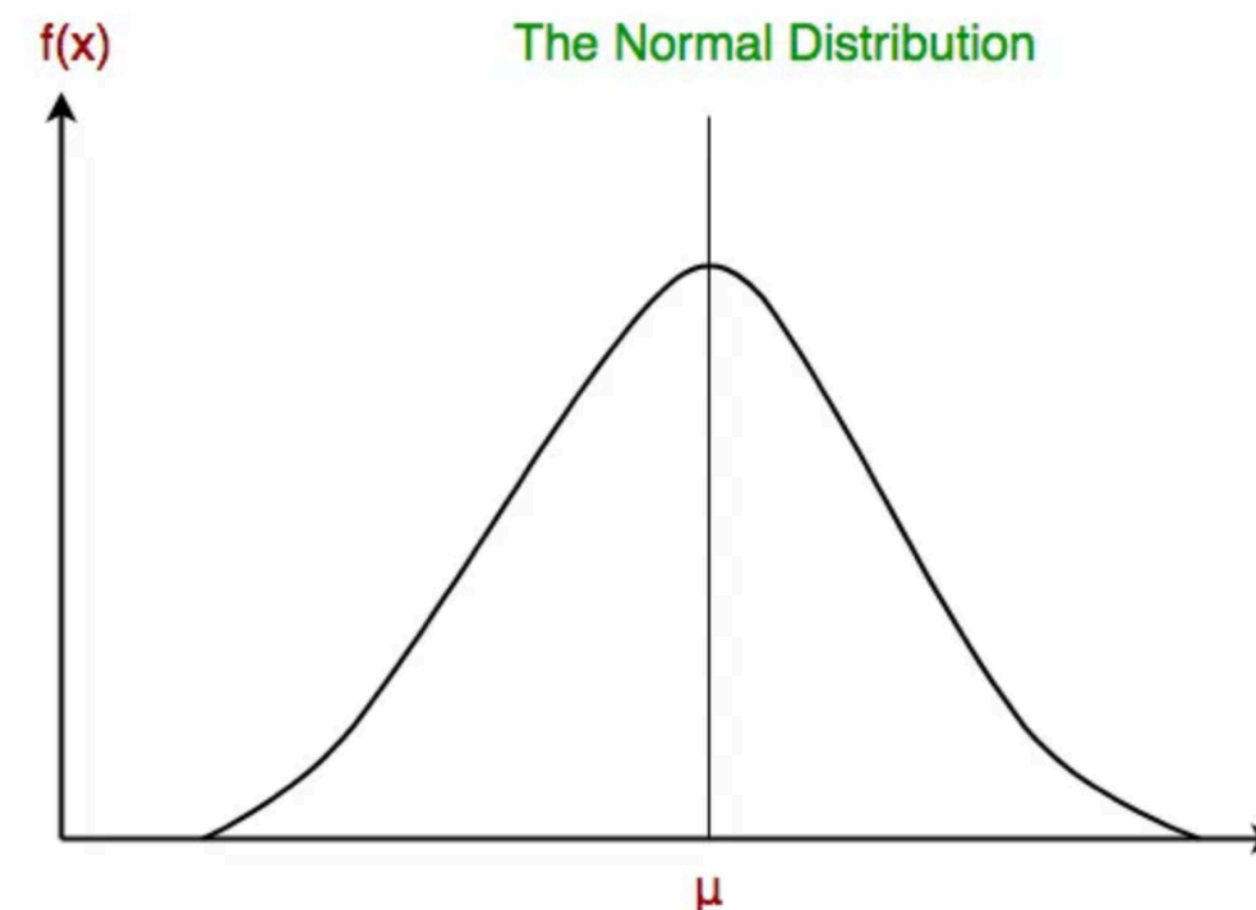
Naive Bayes

Different Flavors

1. Gaussian Naive Bayes (Gaussian NB):

- Works well with **continuous features**.
- Handles data where features can take on **any numerical value within a range, like temperature, height, weight, or sensor readings**.

Assumption: Assumes features follow a **normal distribution** (also known as a Gaussian distribution). This is a **bell-shaped curve** where most data points cluster around the average value.



Naive Bayes

Different Flavors

2. Multinomial Naive Bayes (Multinomial NB):

- It is the most common type and is particularly effective for **text classification**.
- Handles **discrete features**, meaning features represent **counts or frequencies**. These are often integer values but can also be categorical data represented numerically (e.g., word counts in a document, number of times a specific product is purchased by a customer).

Assumption: Assumes features follow a **multinomial distribution**. This means the **probabilities of each feature value (e.g., word frequency) sum up to 1**.

Naive Bayes

Different Flavors

3. Bernoulli Naive Bayes (Bernoulli NB):

- Designed for **binary features**.
- Handles data where **each feature has only two possible values**, typically represented as **0 or 1**. Often used for presence/absence data.

Assumption: Each feature is independent and can be modeled by a Bernoulli distribution.

This means the **probability of each feature value (0 or 1) is independent of other features**.

Naive Bayes

Example

<https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/>

One of my interview question at Apple

Unsupervised Learning

Clustering Techniques

Clustering Techniques

Motivation

- When access to labeled data is limited, clustering algorithms organize a dataset into subsets (***clusters***) where objects in the same cluster are more similar to each other than to those in other clusters.
- **Motivation:** to discover the underlying structure and natural groupings within unlabeled data.
- **Discover hidden patterns:** Clustering is used in the early stages of data analysis to find potential structures or relationships that are not immediately obvious.
- **Simplify complex data:** It reduces the complexity of large datasets by providing an abstraction from individual data objects to the clusters in which they reside.

Clustering Techniques

Business Application

- **Customer segmentation:** Businesses use clustering to divide their customer base into distinct groups with similar behaviors, interests, or demographics. This allows them to create targeted marketing campaigns, improve customer satisfaction, and optimize resource allocation.
- **Recommendation engines:** It can group users with similar tastes to make personalized product or content recommendations.
- **Retail analysis:** Cluster analysis can reveal which items are frequently bought together, informing product placement and promotional strategies.
- **Anomaly and Fraud Detection:** Data points that do not fit into any cluster or are far from their assigned cluster can be flagged as anomalies or outliers.

Clustering Techniques

Research Application

- **Biology and medicine:** It is used to classify patients with similar symptoms or treatment responses, identify different subtypes of a disease, and group genes with similar expression patterns to better understand biological relationships.
- **Climate analysis:** In climate science, clustering can identify weather patterns and trends based on data like atmospheric pressure.
- **Bioinformatics:** Clustering is applied to large biological datasets for tasks like the classification of species.

Clustering Techniques

Formal Definition

we are given a training set $\{x^{(1)}, \dots, x^{(n)}\}$ Here, $x^{(i)} \in \mathbb{R}^d$

- **Objective:** group the data into a few cohesive “clusters.”
- **There is no labeled data**, that means for each $x^{(i)}$ we don't know the $y^{(i)}$

The k -means clustering algorithm is as follows:

1. Initialize **cluster centroids** $\mu_1, \mu_2, \dots, \mu_k \in \mathbb{R}^d$ randomly.

2. Repeat until convergence: {

For every i , set

$$c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2.$$

For each j , set

$$\mu_j := \frac{\sum_{i=1}^n 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^n 1\{c^{(i)} = j\}}.$$

}

← 1. Cluster assignment

← 2. Updating Cluster Centroid

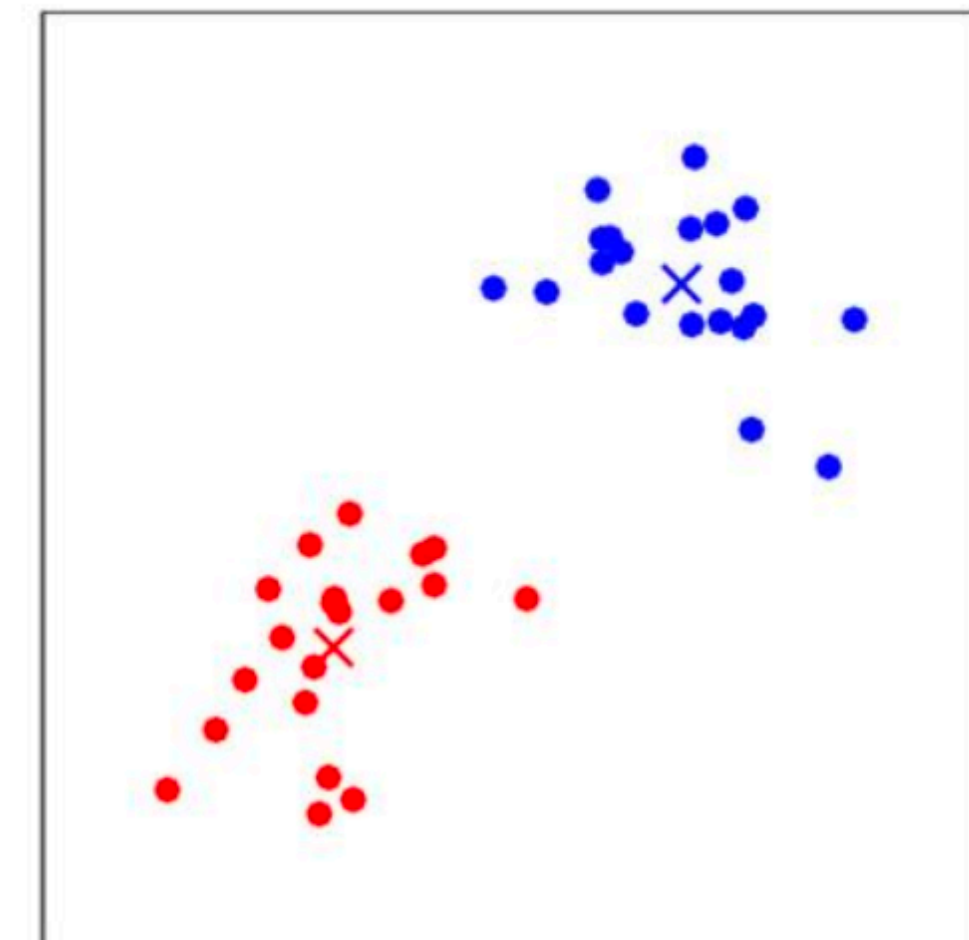
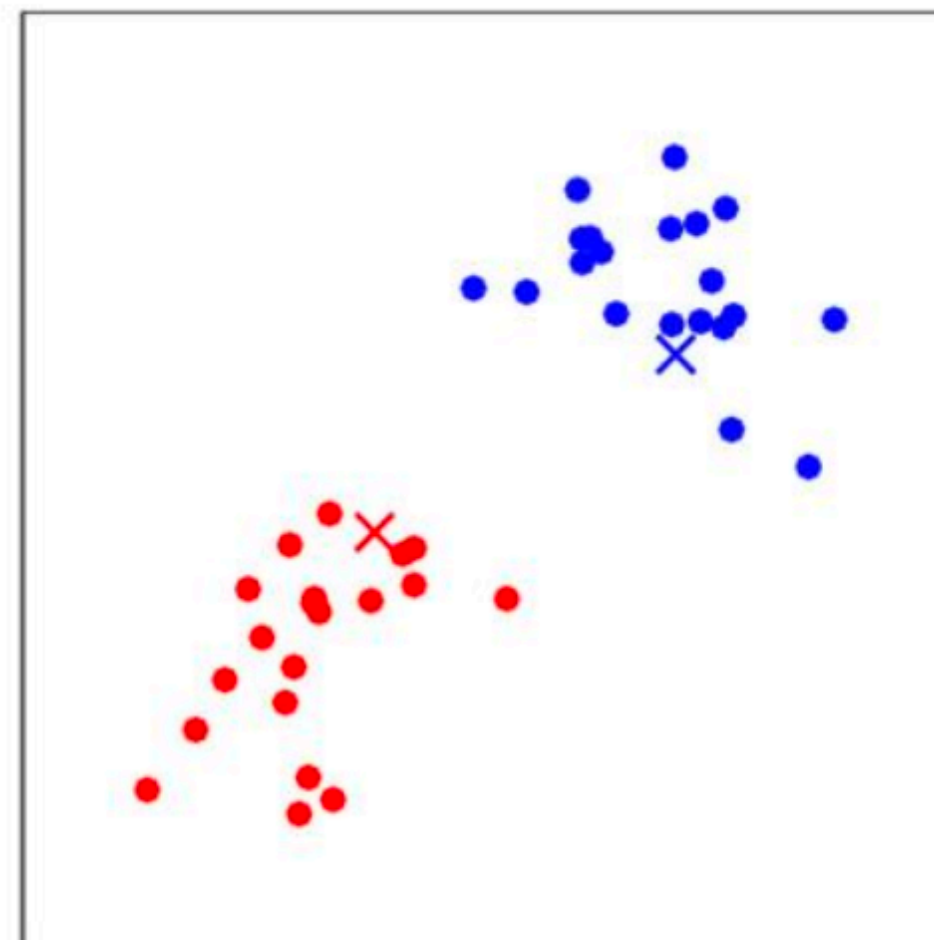
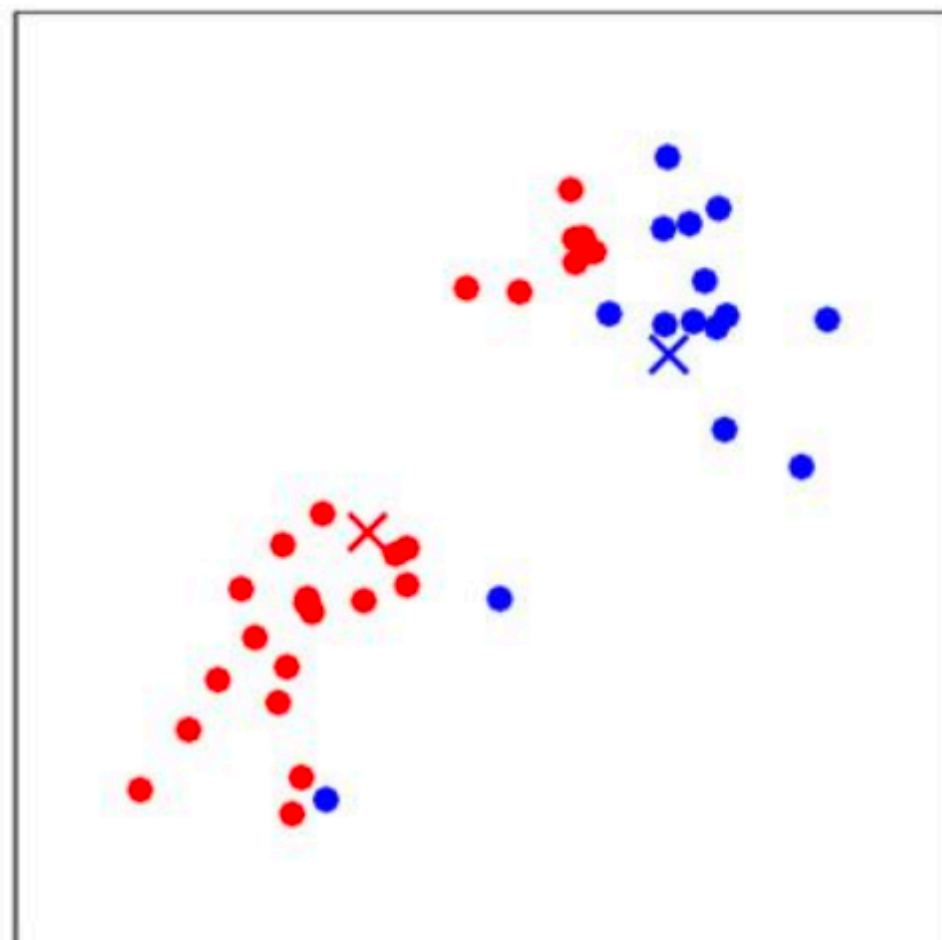
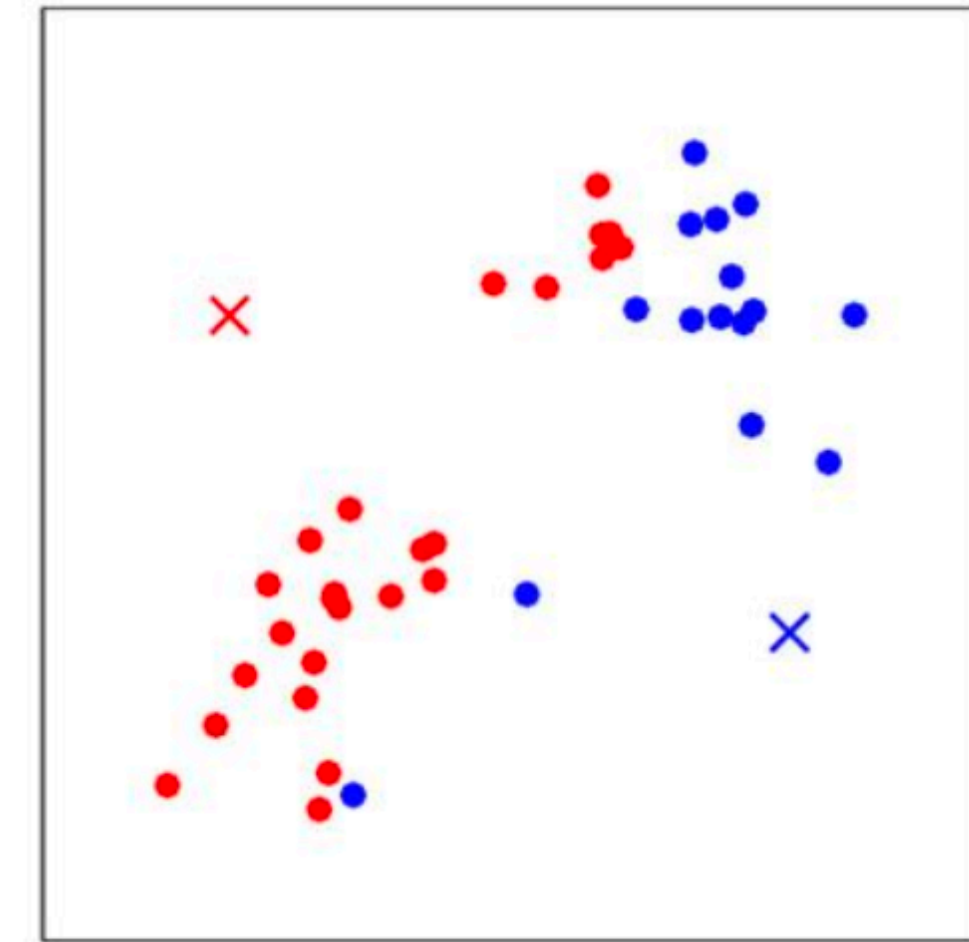
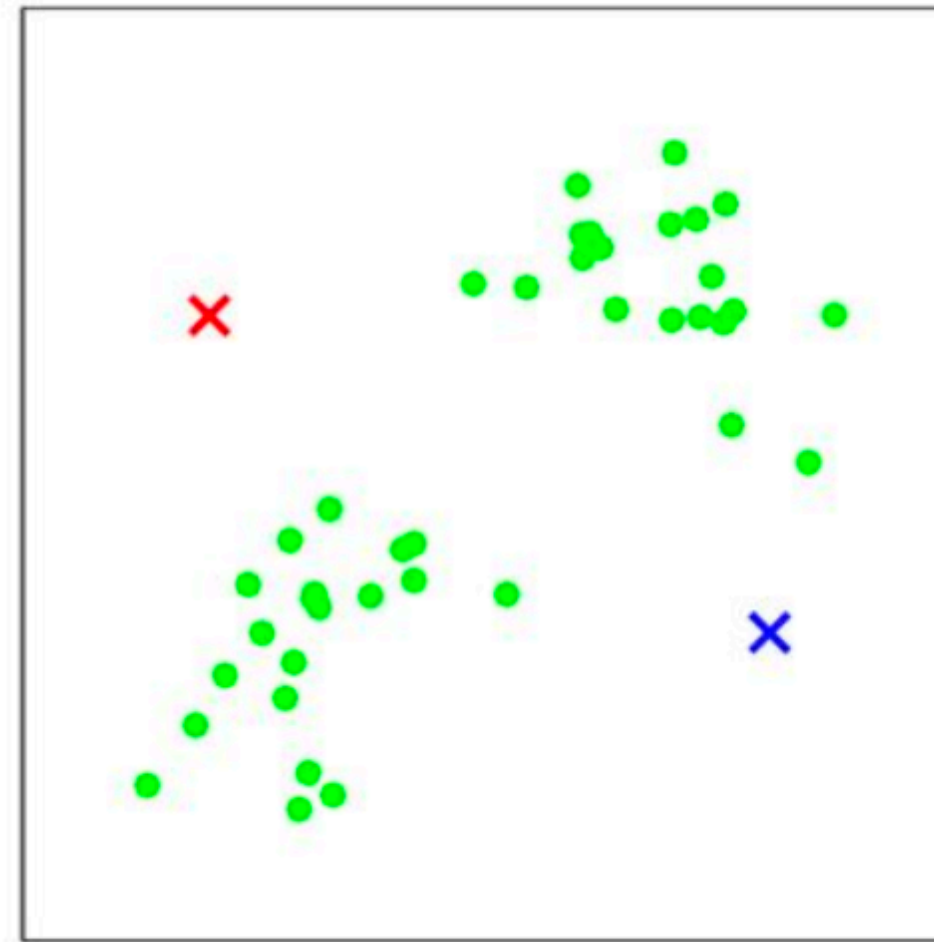
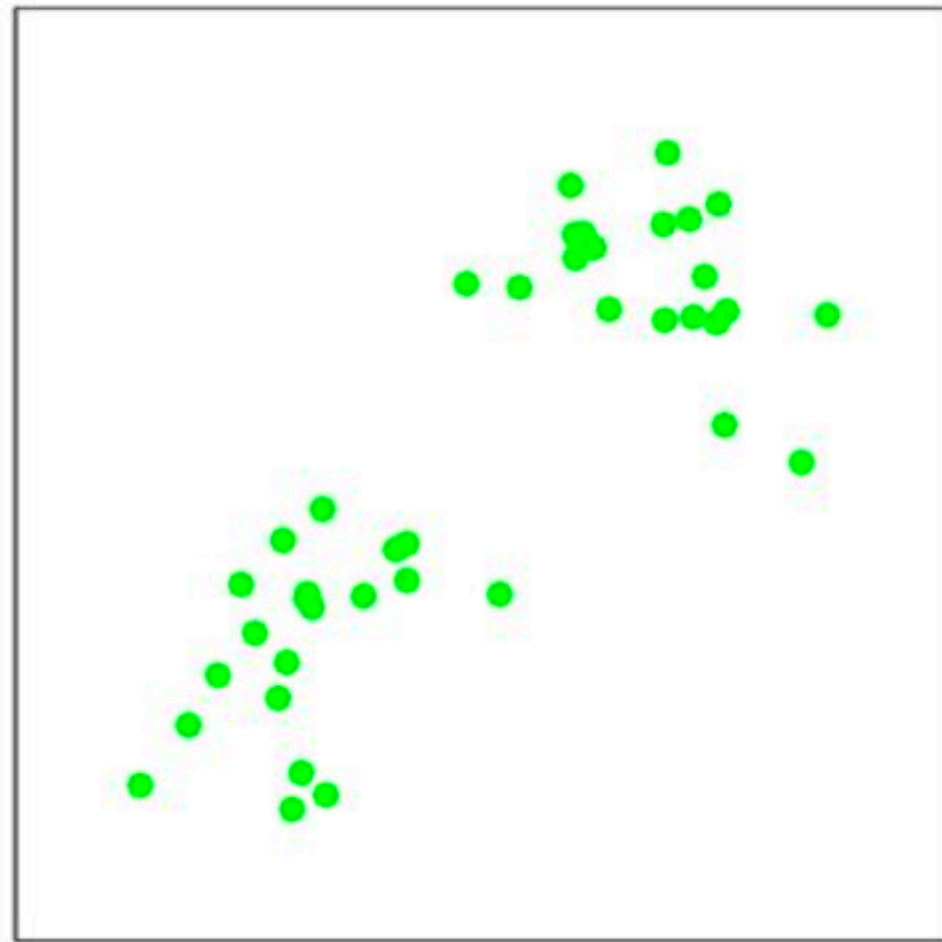
Clustering Techniques

K-means clustering

- k (a parameter of the algorithm) is the number of clusters we want to find.
- **The cluster centroids** μ_j represent our current guesses for the positions of the centers of the clusters.
- **Initialization:**
 - To initialize the cluster centroids (in step 1), we could choose k training examples randomly,
 - Set the cluster centroids to be equal to the values of these k examples.
 - Other initialization methods are also possible, such as PCA-Part method uses Principal Component Analysis (PCA) to find the direction of maximum variance in the data.
- **Objective:** to create clusters where the similarity among objects within a group is high (low intra-cluster distance) and the similarity between objects in different groups is low (high inter-cluster distance).

Clustering Techniques

K-means clustering



Clustering Techniques

K-means convergence

- **Distortion function**

$$J(c, \mu) = \sum_{i=1}^n ||x^{(i)} - \mu_{c(i)}||^2$$

- J (a non-convex function) measures the sum of squared distances between each training example $x^{(i)}$ and the cluster centroid $\mu_{c(i)}$ to which it has been assigned.
- It can be shown that k-means is exactly coordinate descent on J.
- Specifically, the inner-loop of k-means repeatedly minimizes J with respect to c while holding μ fixed, and then minimizes J with respect to μ while holding c fixed.
- Thus, J must monotonically decrease, and the value of J must converge.

Clustering Techniques

K-means convergence

- **Distortion function**

$$J(c, \mu) = \sum_{i=1}^n ||x^{(i)} - \mu_{c(i)}||^2$$

- Thus, J must monotonically decrease, and the value of J must converge.
- **Monotonically decreasing objective function:** The J never increases with each iteration. Each step of the algorithm is designed to either reduce the total squared error or keep it the same.
- **Finite number of states:** For a finite dataset, there is a finite number of possible ways to partition the data points into k clusters.
- K-means is guaranteed to converge to a local minimum, but not necessarily the global minimum.



15 minutes break

Expectation-Maximization Algorithms

Expectation Maximization (EM)

Review Basics

Expectation-Maximization (EM) algorithm is an iterative method for finding maximum likelihood estimates (MLEs) for the parameters of a statistical model, particularly when the model involves unobserved (*latent*) variable.

- $\mathbf{X}$ represents the observed data.
- $\mathbf{Z}$ represents the unobserved or latent variables.
- θ is the vector of unknown parameters of the statistical model.
- $p(\mathbf{X}, \mathbf{Z} | \theta)$ is the complete-data likelihood function, which models the joint probability of both the observed and latent variables.

$$L(\theta | \mathbf{X}) = p(\mathbf{X} | \theta) = \sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z} | \theta)$$

the marginal (or incomplete) likelihood of the observed data, which we want to maximize.

Expectation Maximization (EM) Algorithm

Motivation

- **Motivation:** In clustering, we often assume that the data comes from some underlying probability distribution. For example, in **Gaussian Mixture Models (GMMs)**, we assume that the dataset was generated from a mixture of several Gaussian distributions, each representing a cluster.
- **Challenge:**
 - We don't know the parameters of these distributions (means, variances, mixing proportions).
 - We also don't know which data point came from which cluster (hidden variables).

This is where **Expectation-Maximization (EM)** comes in.

Expectation Maximization (EM) Algorithm

What is EM?

- The **EM algorithm** is a general iterative method to estimate parameters of probabilistic models when the data has hidden/latent variables.
- In clustering, the hidden variable is usually **the cluster assignment** for each data point.
- Suppose that we are given a training set $\{\mathcal{X}^{(1)}, \dots, \mathcal{X}^{(n)}\}$
- We wish to model the data by specifying a joint distribution $p(x^{(i)}, z^{(i)}) = p(x^{(i)} | z^{(i)})p(z^{(i)})$

$$z^{(i)} \sim \text{Multinomial}(\phi) \text{ (where } \phi_j \geq 0, \sum_{j=1}^k \phi_j = 1$$

the parameter ϕ_j gives $p(z^{(i)} = j)$), and $x^{(i)} | z^{(i)} = j \sim \mathcal{N}(\mu_j, \Sigma_j)$

k denote the number of values that the $z^{(i)}$'s can take on

Expectation Maximization (EM) Algorithm

What is EM?

- Suppose that we are given a training set $\{x^{(1)}, \dots, x^{(n)}\}$
- We wish to model the data by specifying a joint distribution $p(x^{(i)}, z^{(i)}) = p(x^{(i)} | z^{(i)})p(z^{(i)})$

$z^{(i)} \sim \text{Multinomial}(\phi)$ (where $\phi_j \geq 0$, $\sum_{j=1}^k \phi_j = 1$)

the parameter ϕ_j gives $p(z^{(i)} = j)$, and $x^{(i)} | z^{(i)} = j \sim \mathcal{N}(\mu_j, \Sigma_j)$

k denote the number of values that the $z^{(i)}$'s can take on

- Our model posits that each $x^{(i)}$ was generated by randomly choosing $z^{(i)}$ from $\{1, \dots, k\}$, and then $x^{(i)}$ was drawn from one of k Gaussians depending on $z^{(i)}$. This is called the ***mixture of Gaussians model***.
- $z^{(i)}$ are ***latent random variables***, meaning that they're hidden/unobserved.

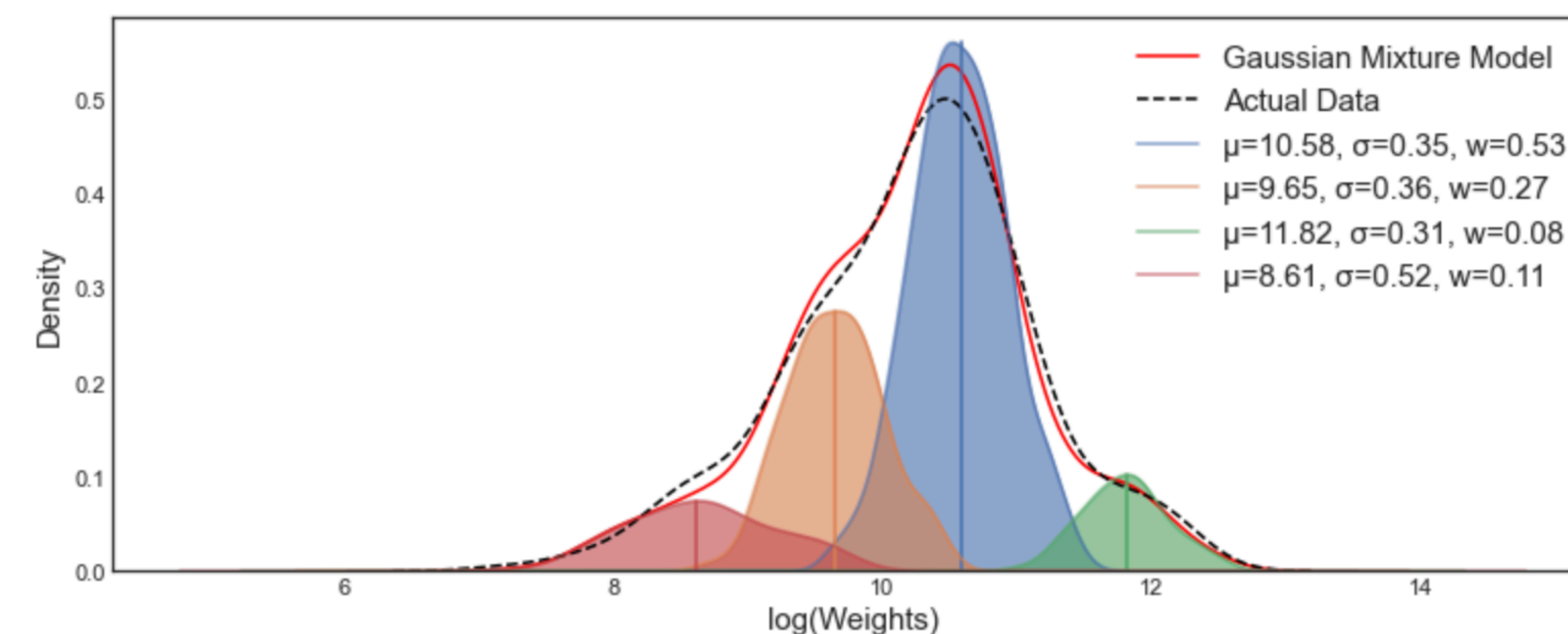
Expectation Maximization (EM) Algorithm

EM in Simple Words

- Suppose that we are given a training set $\{\mathcal{X}^{(1)}, \dots, \mathcal{X}^{(n)}\}$
- We're trying to cluster data points without labels (unsupervised learning).
- To do this, we assume each data point comes from one of several Gaussian bell-shaped distributions (Statistician likes to confuse us so they say this with: *the data was generated by a **mixture of Gaussians***).

$$p(x^{(i)}, z^{(i)}) = p(x^{(i)} | z^{(i)}) p(z^{(i)})$$

- Important things to know:
 - **Hidden variable** $z^{(i)}$
 - **Generating a data point:**
 - **Why it's hard?**



Expectation Maximization (EM) Algorithm

EM in Simple Words

Important things to know:

- **Hidden variable:**
 - Each data point $x^{(i)}$ secretly belongs to some cluster j (but we don't know which one).
 - This hidden cluster assignment is called $z^{(i)}$.
 - We assume $z^{(i)}$ is chosen randomly based on probabilities ϕ (these are the “mixing weights” of the clusters — how much each Gaussian contributes).
- **Generating a data point:**
- **Why it's hard?**

Expectation Maximization (EM) Algorithm

EM in Simple Words

Important things to know:

- **Hidden variable:**
 - Each data point $x^{(i)}$ secretly belongs to some cluster j (but we don't know which one).
 - This hidden cluster assignment is called $z^{(i)}$.
 - We assume $z^{(i)}$ is chosen randomly based on probabilities ϕ (these are the “mixing weights” of the clusters — how much each Gaussian contributes).
- **Generating a data point:**
 - Step 1: Pick a cluster label $z^{(i)}$ according to those probabilities.
 - Step 2: Once a cluster is picked, generate $x^{(i)}$ from that cluster's Gaussian distribution (with its own mean μ_j and covariance Σ_j).
- **Why it's hard?**

Expectation Maximization (EM) Algorithm

EM in Simple Words

- **Hidden variable:**
 - Each data point $x^{(i)}$ secretly belongs to some cluster j (but we don't know which one).
 - This hidden cluster assignment is called $z^{(i)}$.
 - We assume $z^{(i)}$ is chosen randomly based on probabilities ϕ (these are the “mixing weights” of the clusters — how much each Gaussian contributes).
- **Generating a data point:**
 - Step 1: Pick a cluster label $z^{(i)}$ according to those probabilities.
 - Step 2: Once a cluster is picked, generate $x^{(i)}$ from that cluster's Gaussian distribution (with its own mean μ_j and covariance Σ_j).
- **Why it's hard?**
 - We don't observe $z^{(i)}$ (the cluster labels). They're latent variables (hidden).
 - That makes it difficult to estimate the parameters (ϕ, μ_j, Σ_j) .
 - The EM algorithm is designed to solve exactly this kind of problem: ***figuring out parameters when we have hidden variables.***

Expectation Maximization (EM) Algorithm

Simple Analogy for EM

- **Imagine you're at a party, and people brought different types of cookies 🍪:**
 - Chocolate chip
 - Oatmeal raisin
 - Sugar cookie

You get a plate with a random cookie, but you don't know which batch it came from.

- $z^{(i)}$ = which batch the cookie came from (hidden).
- $x^{(i)}$ = the cookie you see (the data).
- Each batch has its own recipe (mean + variance = Gaussian distribution).
- The mixing weights ϕ say how often each type of cookie was baked.

The challenge: from just looking at the cookies (data), can we figure out the recipes (parameters) and how many of each were baked (mixing weights)?

That's the mixture of Gaussians problem.

Expectation Maximization (EM) Algorithm

EM Steps

- Suppose that we are given a training set $\{\mathcal{X}^{(1)}, \dots, \mathcal{X}^{(n)}\}$
- The parameters of our model are (ϕ, μ, Σ) $p(x^{(i)}, z^{(i)}) = p(x^{(i)} | z^{(i)})p(z^{(i)})$
- The likelihood of our data:

$$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^n \log p(x^{(i)} | z^{(i)}; \mu, \Sigma) + \log p(z^{(i)}; \phi)$$

- Maximizing this with respect to (ϕ, μ, Σ) gives the parameters:

$$\begin{aligned}\phi_j &= \frac{1}{n} \sum_{i=1}^n 1\{z^{(i)} = j\}, \\ \mu_j &= \frac{\sum_{i=1}^n 1\{z^{(i)} = j\} x^{(i)}}{\sum_{i=1}^n 1\{z^{(i)} = j\}}, \\ \Sigma_j &= \frac{\sum_{i=1}^n 1\{z^{(i)} = j\} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n 1\{z^{(i)} = j\}}\end{aligned}$$

Expectation Maximization (EM) Algorithm

EM Steps

- Suppose that we are given a training set $\{\mathcal{X}^{(1)}, \dots, \mathcal{X}^{(n)}\}$
- The parameters of our model are (ϕ, μ, Σ) $p(x^{(i)}, z^{(i)}) = p(x^{(i)}|z^{(i)})p(z^{(i)})$
- The likelihood of our data: $\ell(\phi, \mu, \Sigma) = \sum_{i=1}^n \log p(x^{(i)}|z^{(i)}; \mu, \Sigma) + \log p(z^{(i)}; \phi)$
- Maximizing this with respect to (ϕ, μ, Σ) gives the parameters:

If we already knew which cluster each data point belonged to ($z^{(i)}$), then estimating the model's parameters would be just like training a supervised model (like Gaussian discriminant analysis), because the hidden cluster labels would act like class labels.

However, in our density estimation problem, the $z^{(i)}$'s are not known. What can we do?

$$\begin{aligned}\phi_j &= \frac{1}{n} \sum_{i=1}^n 1\{z^{(i)} = j\}, \\ \mu_j &= \frac{\sum_{i=1}^n 1\{z^{(i)} = j\} x^{(i)}}{\sum_{i=1}^n 1\{z^{(i)} = j\}}, \\ \Sigma_j &= \frac{\sum_{i=1}^n 1\{z^{(i)} = j\} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n 1\{z^{(i)} = j\}}\end{aligned}$$

Expectation Maximization (EM) Algorithm

EM Steps

The EM algorithm is an iterative algorithm that has two main steps.

1. **E-Step (Expectation):** it tries to “guess” the values of the $z^{(i)}$'s
2. **M-Step (Maximization):** it updates the parameters of our model based on our guesses.

Expectation Maximization (EM) Algorithm

EM Steps

The EM algorithm is an iterative algorithm that has two main steps.

1. **E-Step (Expectation)**: it tries to “guess” the values of the $z^{(i)}$'s
 - Compute the expected value of the hidden variables (cluster assignment).
 - For GMMs, this means: For each data point $x^{(i)}$, compute the probability that it belongs to each cluster k

For each i, j , set

$$w_j^{(i)} := p(z^{(i)} = j | x^{(i)}; \phi, \mu, \Sigma)$$

2. **M-Step (Maximization)**

Expectation Maximization (EM) Algorithm

EM Steps

The EM algorithm is an iterative algorithm that has two main steps.

1. **E-Step (Expectation):** it tries to “guess” the values of the $z^{(i)}$'s

For each i, j , set

$$w_j^{(i)} := p(z^{(i)} = j | x^{(i)}; \phi, \mu, \Sigma)$$

2. **M-Step (Maximization):**

- Update cluster means:

$$\phi_j := \frac{1}{n} \sum_{i=1}^n w_j^{(i)},$$

- Update covariances:

$$\mu_j := \frac{\sum_{i=1}^n w_j^{(i)} x^{(i)}}{\sum_{i=1}^n w_j^{(i)}},$$

- Update mixing weights:

$$\Sigma_j := \frac{\sum_{i=1}^n w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n w_j^{(i)}}$$

EM vs. K-means Clustering

The EM-algorithm is also reminiscent of the K-means clustering algorithm, except that instead of the “**hard**” cluster assignments $c(i)$, we instead have the “**soft**” assignments $w_j^{(i)}$.

Similar to K-means, it is also susceptible to local optima, so reinitializing at several different initial parameters may be a good idea.

- **K-means** is like saying: “Every point must belong to exactly one cluster, whichever centroid is closest.”
- **EM for GMMs** is like saying: “Every point belongs to all clusters, but with different probabilities, depending on how likely it is under each Gaussian.”

EM vs. K-means Clustering

Feature	K-means	EM (Gaussian Mixture Models)
Type of clustering	Hard clustering → each point belongs to exactly one cluster.	Soft clustering → each point has a probability of belonging to each cluster.
Cluster shape assumption	Assumes clusters are spherical (equal variance, round blobs).	Can model elliptical clusters with different shapes and orientations (via covariance matrices).
Distance measure	Uses Euclidean distance to nearest centroid.	Uses probability densities (e.g., Gaussian likelihoods).
Output	Centroids of clusters.	Means, covariances, and mixing weights of Gaussians.
Flexibility	Simple, fast, but limited.	More powerful, can capture complex cluster shapes.
Computation	Usually faster (fewer parameters).	More computationally expensive (covariance estimation).
Interpretation	No probabilities, just cluster labels.	Probabilistic interpretation (e.g., “this point is 70% in cluster 1, 30% in cluster 2”).

Jensen's Inequality

Intuition

- **Why it's hard?**

$$\ell(\phi, \mu, \Sigma) = \sum_{i=1}^n \log p(x^{(i)} | z^{(i)}; \mu, \Sigma) + \log p(z^{(i)}; \phi)$$

- We don't observe $z^{(i)}$ (the cluster labels). They're latent variables (hidden).
- That makes it difficult to estimate the parameters (ϕ, μ_j, Σ_j) .
- The EM algorithm is designed to solve exactly this kind of problem: *figuring out parameters when we have hidden variables*.
- **Jensen's inequality** comes up a lot in probability, statistics, and especially when we talk about the EM algorithm.
- **In EM algorithm:** Jensen's inequality is used to create a **lower bound** on the log-likelihood. This lower bound makes optimization tractable.
- **In deep learning / variational inference:** It's the basis for methods like ELBO (Evidence Lower Bound) in variational auto-encoders.

Jensen's Inequality

Convex Function

A function $f(x)$ is **convex** if its graph curves **upward** like a bowl ($\cup$).

For any two points x_1, x_2 and any $\lambda \in [0, 1]$:

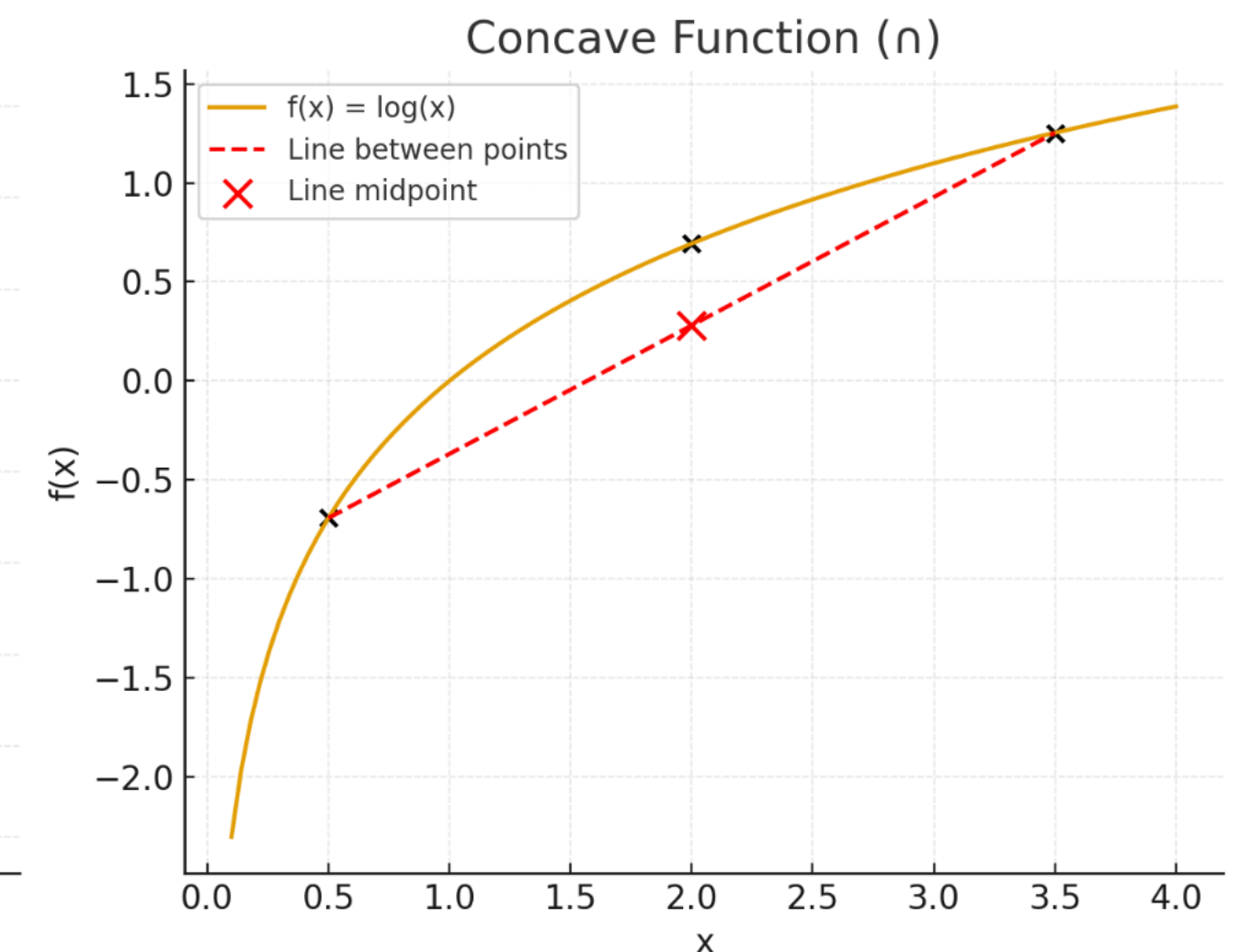
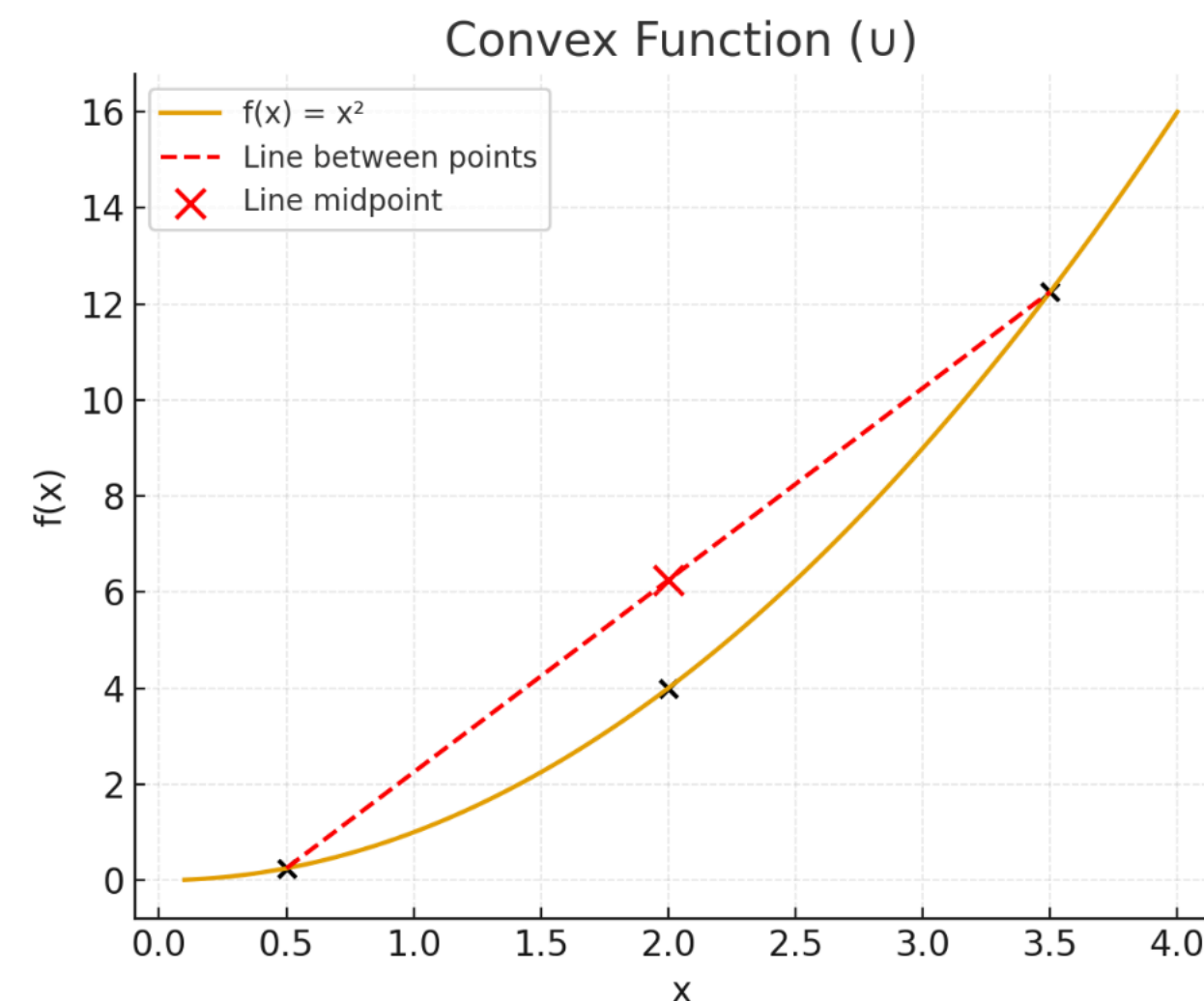
$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

Concave Function

A function $f(x)$ is **concave** if its graph curves **downward** like a bowl ($\cap$).

For any two points x_1, x_2 and any $\lambda \in [0, 1]$:

$$f(\lambda x_1 + (1 - \lambda)x_2) \geq \lambda f(x_1) + (1 - \lambda)f(x_2)$$



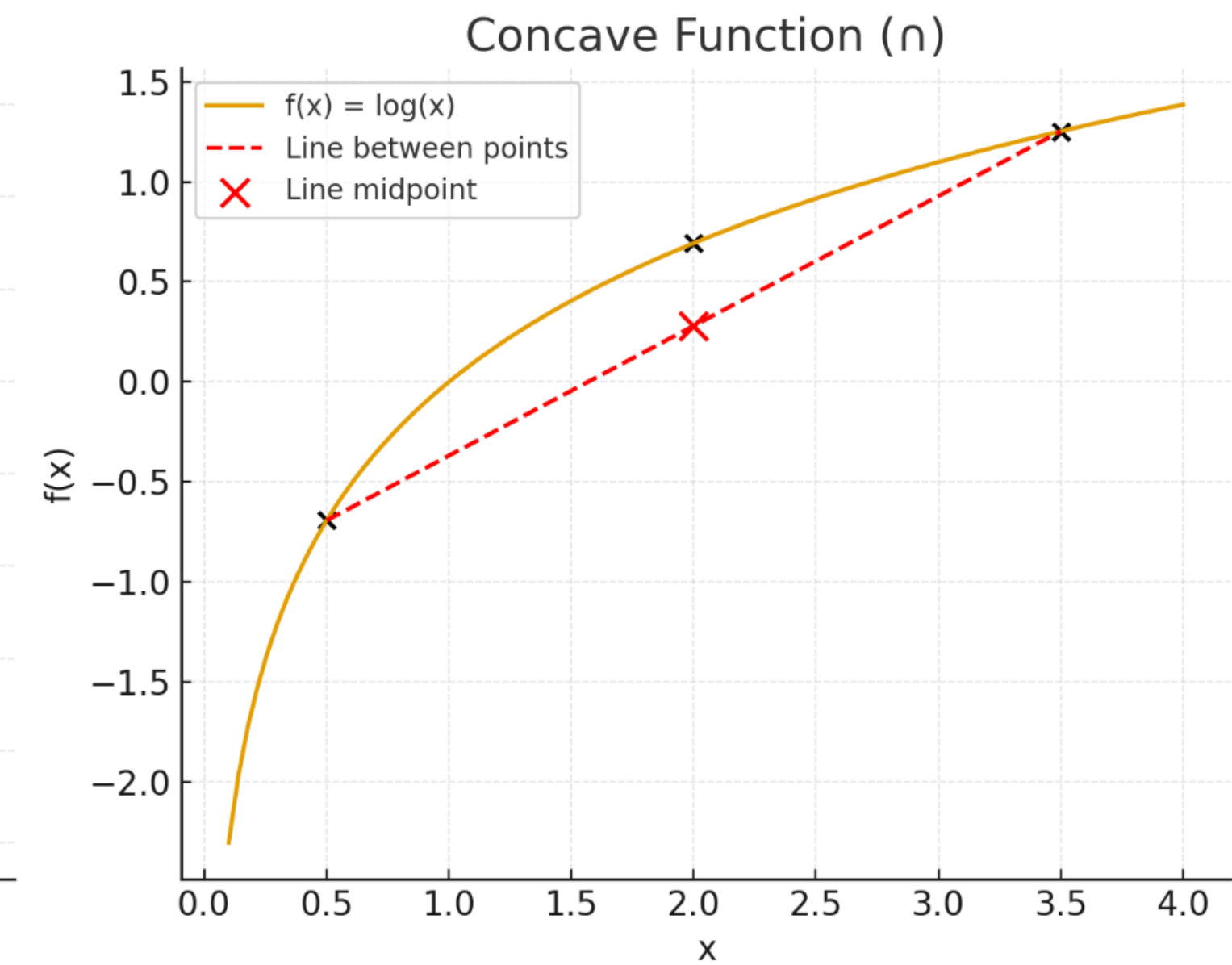
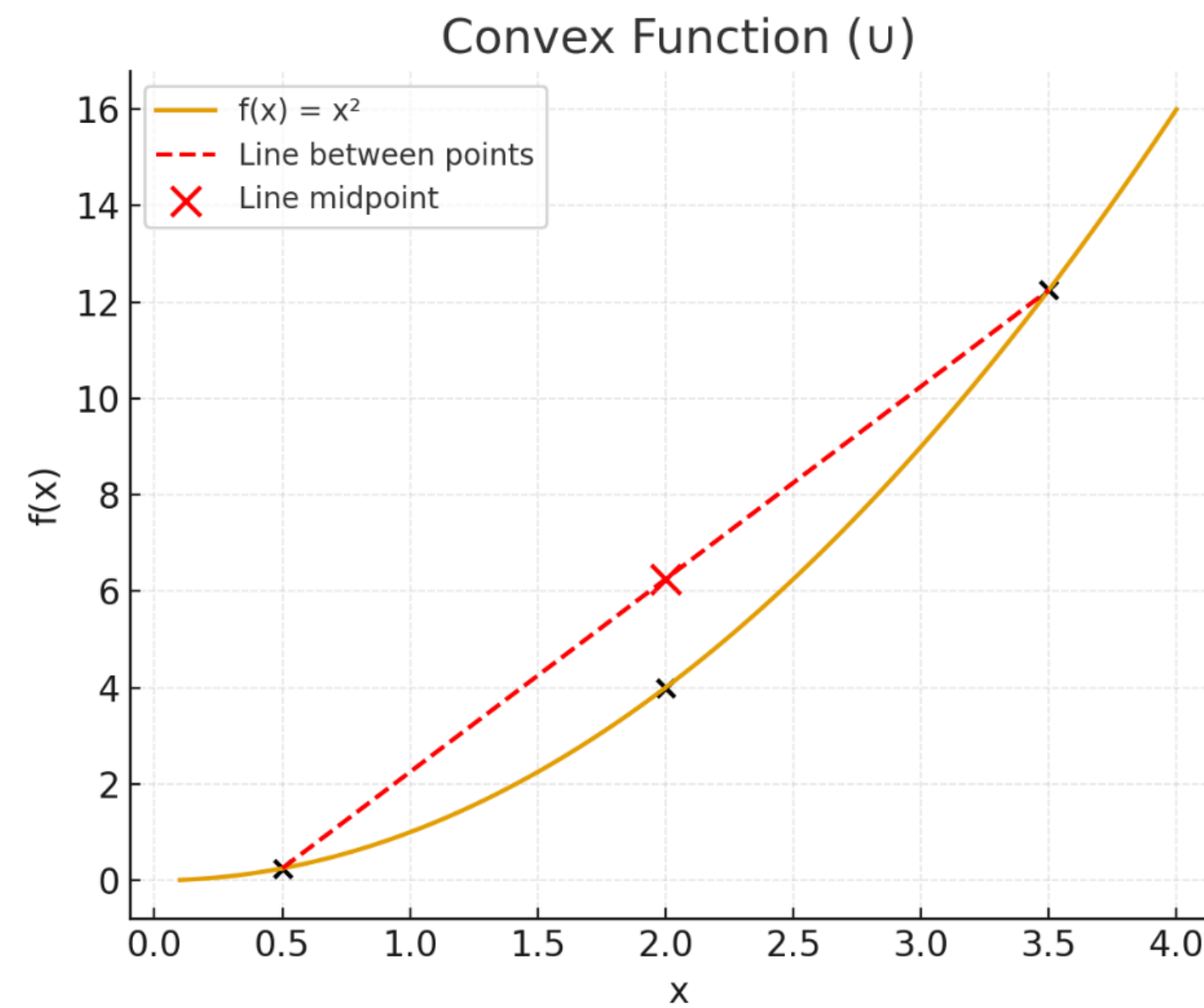
Jensen's Inequality

A function $f(x)$ is **convex** if its graph curves **upward** like a bowl ($\cup$).

A function $f(x)$ is **concave** if its graph curves **downward** like a bowl ($\cap$).

In convex optimization, the fact that the curve lies below the line means
“**any local minimum is also a global minimum.**”

That's why convex functions are “nice”
they have no tricky local minima.



Jensen's Inequality

Formal Statement

For a **convex** function f and a random variable X :
$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$$

If f is **concave**, the inequality flips:
$$f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$$

Intuition:

Convex functions **curve upward**. Because of this curve, if you take the average first and then apply the function, you get a smaller value than if you apply the function first and then average.

Why it matters?

- **In EM algorithm:** Jensen's inequality is used to create a **lower bound** on the log-likelihood. This lower bound makes optimization tractable.
- **In deep learning / variational inference:** It's the basis for methods like ELBO (Evidence Lower Bound) in variational autoencoders.

Evidence Lower Bound (ELBO)

ELBO Methods

EM Recap

In EM, we want to maximize the **log-likelihood** of the data $p(x; \theta) = \sum_z p(x, z; \theta)$

But this log-likelihood is **hard to compute directly** because it involves hidden/latent variables Z .

EM uses **Jensen's inequality** to form a lower bound on the log-likelihood.

Let Q be a distribution over the possible values of z . That is, $\sum_z Q(z) = 1, Q(z) \geq 0$

That bound depends on a distribution $q(Z)$ over the hidden variables:

$$\underbrace{\log p(X \mid \theta)}_{\text{ELBOW: Lower bound}} \geq \mathbb{E}_{q(Z)}[\log p(X, Z \mid \theta)] - \mathbb{E}_{q(Z)}[\log q(Z)]$$

ELBOW: Lower bound

ELBO Methods

Formal definition

In EM, we want to maximize the **log-likelihood** of the data

$$p(x; \theta) = \sum_z p(x, z; \theta)$$

Let Q be a distribution over the possible values of z . That is, $\sum_z Q(z) = 1, Q(z) \geq 0$

That bound depends on a distribution $q(Z)$ over the hidden variables:

$$\log p(X \mid \theta) \geq \mathbb{E}_{q(Z)} [\log p(X, Z \mid \theta)] - \mathbb{E}_{q(Z)} [\log q(Z)]$$

$$\text{ELBO}(q, \theta) = \mathbb{E}_{q(Z)} [\log p(X, Z \mid \theta)] - \mathbb{E}_{q(Z)} [\log q(Z)]$$

Maximizing ELBO is equivalent to making $q(Z)$ (our guess of the latent variable distribution) close to the true posterior $p(Z \mid X, \theta)$.

ELBO Methods in Deep Learning

Impact

1. **Variational Inference (VI):** In complex models (e.g. neural networks with latent variables), the true posterior $p(Z|\mathcal{X}, \mathcal{D})$ is intractable. So we pick a family of distributions $q_{\phi}(Z)$ (parameterized by neural networks) and maximize ELBO instead.
2. **Variational Autoencoders (VAEs):** VAEs are built entirely on ELBO.
Encoder = $q_{\phi}(Z|\mathcal{X})$ (approx posterior).
Decoder = $p_{\theta}(\mathcal{X}|Z)$.
Training objective = maximize ELBO:
3. **General probabilistic deep learning:** ELBO is the foundation for Bayesian deep learning, probabilistic graphical models, and many generative models (e.g., diffusion, latent variable models).

ELBO Methods in Deep Learning

Impact

- EM showed us how to handle latent variables via a lower bound.
- ELBO generalizes this idea, enabling neural network–based variational inference.
- Without ELBO, VAEs, probabilistic deep learning, and modern Bayesian approaches would not be possible.
- Read Chapter 11.3 General EM algorithms from CS299MainNotes.

Beyond GMM Models

Mixture of Different Distributions

GMMs assume clusters are Gaussian (bell-shaped).

But in many real-world problems, data doesn't follow a Gaussian.

Instead, you can model mixtures of other distributions:

Mixture of Bernoullis:

- For binary data (e.g., pixel on/off in black-and-white images).
- Each cluster is defined by a Bernoulli probability vector.

Mixture of Multinomials:

- For categorical/count data (e.g., text documents represented as word counts).
- This leads directly to **topic models** like LDA (Latent Dirichlet Allocation).

Mixture of Poissons:

- For count data (e.g., number of website visits, accidents, or calls per day).

Robust Mixture Models

Mixture of t-distributions:

- Heavier tails than Gaussians → more robust to outliers.
- Useful in finance or biological data where outliers are common.

Nonparametric mixtures:

- Replace fixed distributions with flexible ones like kernel densities.
- Capture more irregular cluster shapes.

Bayesian Mixture Models

Standard GMM requires specifying K (number of clusters).

But Bayesian approaches let the data “decide”.

- **Dirichlet Process Mixture Models (DPMMs)**
 - Infinite mixture model where the number of clusters grows with data.
 - Uses a **Dirichlet Process prior** on cluster assignments.
 - Practical for situations where K is unknown.
- **Example — Restaurant Process:**
 - Imagine customers (data points) entering a restaurant and sitting at tables (clusters).
 - A new customer either joins an existing table with probability proportional to its size or starts a new table.
 - This creates a flexible, data-driven number of clusters.

Applications

Speech recognition: GMM-HMMs were the workhorse before deep learning took over.

Image segmentation: mixture models for separating colors/textures.

Genetics: mixture models for population clustering.

Text mining: multinomial mixtures $\rightarrow$ topic models.

Reading List

1. <https://towardsdatascience.com/the-math-and-code-behind-k-means-clustering-795582423666/>
2. <https://www.cs.toronto.edu/~mbrubake/teaching/C11/Handouts/Clustering.pdf>
3. https://www.mathwords.com/j/jensens_inequality.htm
4. <https://mathmonks.com/inequalities/jensens-inequality>
5. <https://arxiv.org/html/2503.19618v1>



Questions?

Email: zara.hajihashemi@sjsu.edu